Quantum Information Processing (2023) 22:138
https://doi.org/10.1007/s11128-023-03856-y

On new PageRank computation methods using quantum computing

Théodore Chapuis-Chkaiban[1] · Zeno Toffano[2] · Benoît Valiron[3]

Received: 15 January 2022 / Accepted: 20 January 2023 / Published online: 8 March 2023

Abstract

In this paper we propose several new quantum computation algorithms as an original contribution to the domain of PageRank algorithm theory, Spectral Graph Theory and Quantum Signal Processing. We first propose an application to PageRank of the HHL quantum algorithm for linear equation systems. We then introduce one of the first Quantum-Based Algorithms to perform a directed Graph Fourier Transform with a low gate complexity. After proposing a generalized PageRank formulation, based on ideas stemming from Spectral Graph Theory, we show how our quantum directed graph Fourier Transform can be applied to compute our generalized version of the PageRank.

Keywords Quantum computing · PageRank · Graph Fourier transform · Spectral graph theory · Information theory

Zeno Toffano and Benoît Valiron contributed equally to this work.

✉ Théodore Chapuis-Chkaiban
theodore.chapuis-chkaiban@student-cs.fr

Zeno Toffano
zeno.toffano@centralesupelec.fr

Benoît Valiron
benoit.valiron@centralesupelec.fr

[1] Department of Computer Science, CentraleSupélec, 3 Rue Joliot-Curie, 91190 Gif-Sur-Yvette, France

[2] CNRS, CentraleSupélec, Laboratoire des signaux et systèmes, Université Paris-Saclay, 91190 Gif-sur-Yvette, France

[3] CNRS, ENS Paris-Saclay, CentraleSupélec, Laboratoire Méthodes Formelles, Université Paris-Saclay, 91190 Gif-sur-Yvette, France

1 Introduction

The Google PageRank algorithm was introduced by Sergey Brin and Lawrence Page in their paper entitled "The anatomy of a large-scale hypertextual Web search engine" [1]. The PageRank algorithm produces a node ranking score by order of importance on a directed graph. The algorithm structured the Web: although it is now integrated into a larger pipeline, it still remains a major component of Google's search engine.

Numerous applications have been derived. For instance in social networks, this algorithm can be used to sort the most influential users or contents. Use cases can be found in various fields like sports, literature, neuroscience, toxic waste management, debugging and road traffic prediction. See [2] for a broad discussion and [3] where David F. Gleich discusses extensively the various applications of the PageRank algorithm.

On the mathematical side the algorithm relies on the Perron–Frobenius theorem producing an approximation of a final PageRank vector. In an unparallel setting, the PageRank of a n-node graph can be computed in $\mathcal{O}(n)$ operations (it amounts to do a constant number of sparse matrix–vector multiplications). The computational complexity is logarithmic on the precision one wants to achieve on the final rankings.

Spectral Graph Theory (SGT) is a new mathematical theory used to adapt signal processing to graph theory. Graph Fourier Transform—a major tool used in SGT—provides interesting results on the properties of directed graphs [4]. This theory has numerous applications and consequences on the study of graphs: from Machine Learning and Computer Graphics to pure mathematics. See [4–7] for instance.

Three original proposals are presented in this paper:

- We propose an efficient way to compute the classical PageRank using quantum circuits by using the HHL (Harrow, Hassidim, Lloyd) algorithm [8]. We point out that the classical PageRank algorithm can be efficiently computed using the HHL algorithm. To our knowledge this is the first quantum circuit-based PageRank algorithm and can be seen as the equivalent of the Quantum Adiabatic Algorithm [9] with some minor refinements on the treatment of dangling nodes and squared state amplitudes. Our algorithm may be seen as a solution to the open problem stated at the end of [9]. Our version achieves a better theoretical complexity in the final state preparation and improves the number of samples needed to approximate the top ranking values. As far as we know, this theoretical logarithmic complexity has never been reached for the PageRank algorithm, the previous best result was obtained by [9], with a $\mathcal{O}(polylog(n))$ experimental complexity.
- We then propose a quantum computing framework for Spectral Graph Theory. We adapt the Quantum Singular Thresholding Algorithm [10] to compute inverse Graph Fourier Transforms. We show that, provided one can access to an efficient quantum data structure, one can either perform advanced graph signal filtering or produce complex signals based on inverse Graph Fourier Transform using a number of quantum gates that scales linearly with the degree of a polynomial approximation of the filter.
- We finally generalize Google's classical PageRank algorithm and provide a new PageRank formalism that can be used as well in classical and quantum Signal Pro-

cessing. We then show that the Quantum Singular Value Transformation (QSVT) algorithm can compute this generalized version of the PageRank using quantum circuits. The method relies again on Spectral Graph Theory and produces rankings by the investigation of the input Graph transition matrix eigenspaces. We provide numerical results refining the ranking obtained by the classical PageRank algorithm, respecting the original structure of the input graph. We then show that our generalized PageRank method can be efficiently implemented on a quantum computer using our Quantum Signal Processing Algorithm. We also draw a parallel between our method and G. Paparo's discrete PageRank quantum algorithm [11].

We have divided this article into 6 sections:

1. Introduction
2. General notions on PageRank algorithm theory and state of the art of the quantum approaches of PageRank.
3. HHL algorithm application to PageRank.
4. Quantum algorithm For Graph Fourier Transform computation
5. Application of Spectral Graph Theory to PageRank
6. Conclusion

We added an appendix that details numerical examples of our Generalized PageRank using various Kernels.

2 PageRank theory and quantum computation

A directed, finite weighted graph $\mathcal{G}$ is a triple $(V, E, \mathbf{W}(E))$ where V is a set of vertices, $E \subseteq V \times V$ is a set of pairs of vertices and $\mathbf{W} : V \times V \to \mathbb{R}^+$ is a map such that

$$\forall (i,j) \in V \times V, \quad \begin{cases} \mathbf{W}(i,j) > 0 & \text{if } (i,j) \in E \\ \mathbf{W}(i,j) = 0 & \text{otherwise} \end{cases}$$

In this paper, we only consider finite graphs, i.e., $|V| < \infty$. Furthermore, we consider *normalized* directed weighted graphs: for all $i \in V$, $\sum_{j \in V} \mathbf{W}(i,j)$ is either 1 (if i is the source of some edge) or 0 if i only admits incoming edges.

Throughout this paper, we shall be using the simpler terms *directed graphs* or *graphs* in place of *normalized, directed finite weighted graph*. The finite and weighted properties are implicit if not precise otherwise. Similarly, the terms *weighted adjacency matrix* and *adjacency matrix* will be used indistinctly.

The PageRank algorithm inputs a directed graph and outputs an *importance vector* $\mathbf{I} : V \to \mathbb{R}$, that establishes a ranking of the nodes by order of "importance", notion to be discussed in Sect. 5

While the original PageRank algorithm can have any type of graph as input, in this paper we focus on *scale-free networks*: graphs whose degree distribution follows an asymptotic power law. Such a law asks for the proportion *P(k)* of nodes having k outgoing edges to be $P(k) \simeq k^{-\gamma}$. Graphs modeling human-based systems such as the Internet or Social Networks typically fall in this category, with coefficient values $2 \leq \gamma \leq 3$ [12–14].

2.1 Perron-Frobenius and limit distributions

The PageRank algorithm is based on Random Walks, more specifically on Markov Chains—it uses the Perron Frobenius theorem to compute the importance vector. The intuition is that the adjacency (weighted) matrix of a directed graph $\mathcal{G}$ can be identified with the transition matrix of a Markov chain.

The transition matrix of a Markov Chain on a graph that doesn't have any *dangling nodes* (i.e., nodes that has no outgoing edges) is *left stochastic* (i.e., the coefficients of each row sum to one). If the graph admits dangling nodes, then the transition matrix has a null row.

A Markov Chain C associated with the probability transition matrix $\mathbf{P}$ is regular if and only if some power of $\mathbf{P}$ only has positive entries. Note that a regular matrix is necessarily left-stochastic—and hence is not associated with a graph that admits dandling nodes. Regularity is equivalent to the conjunction of the two following properties:

1. connectivity: $\forall(i, j), \exists\, k$ so that: $\mathbf{P}^k(i, j) > 0$,
2. aperiodicity: $\forall i$: $gcd\{k : \mathbf{P}^k(i, i) > 0\} = 1$.

We say that a node i has a *period* k if, starting from i, a random walker can only get back to i by a multiple of k steps. The *period* of a graph is the greatest common divisor (gcd) of the periods of its nodes. A bipartite graph, for instance, is a 2-periodic graph. A graph is aperiodic if it is a 1-periodic graph.

The Perron–Frobenius Theorem makes it possible to relate eigenvectors of the transition matrix of a Markov Chain and its limit distribution.

Definition-Theorem 2.1 (Perron–Frobenius Theorem for Positive Matrices [15]) *Let $\boldsymbol{M} \in \mathcal{M}_n(\mathbb{C})$ be a regular matrix. Then the value* 1 *is a right eigenvalue of $\boldsymbol{M}$, and any other (right) eigenvalue λ of $\boldsymbol{M}$ has an absolute value strictly smaller than 1. The right eigenspace associated to* 1 *is one-dimensional and contains a vector with only positive coefficients, summing to* 1. *This vector is called the (right) Perron-vector of $\boldsymbol{M}$. The same result applies for* left *eigenvalues (thus getting a* left *Perron-vector).*

Hence, given a regular Markov Chain C associated with a regular matrix $\mathbf{M}$, the probability limit distribution of C, is exactly the left Perron-vector of $\mathbf{M}$.

2.2 Patching the dangling nodes

Let $\mathcal{G} = (V, E)$ be a directed graph with transition matrix $\mathbf{P}$. If the graph $\mathcal{G}$ features dandling nodes, one transforms the matrix $\mathbf{P}$ into $\bar{\mathbf{P}}$ where

$$\bar{\mathbf{P}}_i \triangleq \begin{cases} \mathbf{P}_i & \text{if } i \text{ is not a dandling node} \\ (\rho_1, \ldots, \rho_n) & \text{if } i \text{ is a dangling node} \end{cases}$$

And where $\bar{\mathbf{P}}_i$ is the ith row of $\bar{\mathbf{P}}$ and $(\rho_1, \ldots, \rho_n)$ is some chosen row stochastic vector (i.e., $\sum_{i \in [1,n]} \rho_i = 1$). We call the matrix $\bar{\mathbf{P}}$ a dangling-node free transition matrix of C.

This adds virtual outgoing edges to prevent input graphs from having dandling nodes. Indeed, otherwise it is impossible to escape from their attraction: once a random walker is trapped into a dangling node, he cannot move to another node and the limit probability distribution admits only one nonzero coefficient.

Patching a matrix $\mathbf{P}$ to a dangling-free version $\bar{\mathbf{P}}$ requires the choice of a vector ρ. In [16] an extensive discussion is presented on the subject defining three types of patches: *Strongly Preferential PageRank*—taking ρ as the uniform probability distribution [3], *Weakly Preferential PageRank* and *Sink Preferential PageRank*—which are less common [3].

2.3 Google matrix

In Lawrence Page & Sergey Brin's paper [1], the Google matrix associated to a graph $\mathcal{G}$ is defined as

$$\mathbf{G} \triangleq (1-\alpha)\bar{\mathbf{P}} + \frac{\alpha}{N}\mathbf{J},$$

where $\bar{\mathbf{P}}$ is a dandling-free transition matrix associated to $\mathcal{G}$, and where $\mathbf{J}$ is the matrix that only contains ones and α is a constant ($\alpha \in [0, 1]$).

The original intuition behind the Google matrix is the following: let $\mathcal{G} = (V, E)$ be a directed graph representing the relations between the web Pages—i.e., where $x \in V$ represents a Web Page and $(i, j) \in E$ is the link from i pointing to j. In order to quantify the "importance" of the nodes of the Graph we propagate a random walker on the Web and compute the probability that the Walker gets to a given node after a long time: the higher the probability the more relevant the node.

The Google matrix transforms the input graph by adding low-weighted edges between any pairs of nodes, making the input graph strongly connected (and thus regular). Google uses the value of $\alpha = 0.15$ for its web search engine: this is an heuristic choice to balance fast convergence and pertinence of the results [17]. In fact, the smaller the constant α is, the better the structure of the original graph is preserved, while the greater α is, the faster the PageRank converges.

The *Google importance vector* is the Perron-vector of the Google Matrix of $\mathcal{G}$. In other words, the Google importance vector is defined as being the only unit left row eigenvector associated to the eigenvalue 1 of the Google Matrix.

2.4 Classical Google PageRank

The classical PageRank algorithm aims at finding the Google importance vector out of a Google matrix. It proceeds as follows.

1. Choose a starting (row) vector $\mathbf{x}$ (for instance, pick $\mathbf{x} = \mathbf{j}/N$ where $\mathbf{j}$ is the vector filled with ones and n is the number of nodes in the graph).
2. Compute $\mathbf{y} := \mathbf{x}\mathbf{G}$.
3. If $\|\mathbf{x} - \mathbf{y}\| \geq \epsilon$, perform the update $\mathbf{x} := \mathbf{y}$ and jump back to Step (1).
4. Otherwise, output $\mathbf{y}$.

There are various ways to reduce the computational cost of the update operation. Indeed, the input transition matrix $\mathbf{P}$ is sparse and the Google matrix is a linear combination of $\mathbf{P}$ and a constant matrix: the update operation then scales as $\mathcal{O}(nnz(\mathbf{P})) \simeq \mathcal{O}(n)$ where $nnz(\mathbf{P})$ is the number of non-zero components of $\mathbf{P}$. See [17] for details on the explicit computation and optimization of the classical PageRank algorithm.

2.5 Quantum PageRank

Over the past years, two main directions have been taken to adapt the PageRank algorithm to the Quantum domain.

The first direction was taken in [9], proposing polynomial computational improvements over its classical counterpart, but this method does not use Quantum Walks to improve the ranking performance.

A second one, [11, 18–21], studies the influence of Quantum Random Walks on the PageRank algorithm rankings. This approach exploits Quantum Walk properties such as quantum state interference, which heavily modifies the walker probability distribution on the state graph [22] and reveals the graph structure more precisely [18, 19, 21]. These algorithms are essentially designed to produce classical simulations of discrete quantum walks. Hence, these approaches do not provide significant improvements compared to the solutions based on classical random walk.

3 Quantum PageRank using the HHL algorithm

Here we present our first application: adapting the quantum adiabatic algorithm for search engines introduced by Gamerone, Zanardi and Lidar in 2012 [9] to PageRank. The linear system formulation of PageRank indeed possesses several interesting properties that makes it particularly suitable to be solved by the HHL algorithm [8]. Our contribution presents a quantum circuit algorithm that prepares a quantum state corresponding to the Google importance vector. To our knowledge this application has never been proposed so far, and it has multiple interesting applications (some of them inspired from [9]).

3.1 Problem statement

3.1.1 Inputs

The personalized Google matrix is used as input:

$$\mathbf{G} = \alpha\mathbf{P} + (1-\alpha)\mathbf{j}\mathbf{v}^T$$

Here, $\mathbf{j}$ is the constant vector of $\mathbb{R}^N$ containing only ones, $\mathbf{j} = (1, 1, \ldots, 1)$, $\mathbf{P}$ is the stochastic transition matrix of a dangling node free graph, α is a constant and $\mathbf{v} \in \mathbb{R}^+$ is the personalization vector. The personalization vector removes dangling

nodes from **G** 2.2 and introduce a bias on the jump location of the random walker (note that for Google's matrix $\mathbf{v} = (1/N, \ldots, 1/N)$, where N is the number of nodes in the graph. This formulation encapsulates a more general category of PageRank problems that includes the classical PageRank formulation given in 2.4, but also the preferential PageRank [3].

For a correct performance of our algorithm, **P** must be a *s-sparse* matrix. To satisfy this requirement, the dangling node removal procedure must not produce dense rows in the dangling free transition matrix. For this purpose, we choose to follow the *Weak Preferential PageRank* methods when handling with dangling nodes:

- *Sink Preferential PageRank*: we apply the transformation $\mathbf{P} \rightarrow \mathbf{P} + diag(c_i)$, where $c_i = 1 iff i$ is a dangling node. This amounts for the walker to wait on the dangling node with probability α, and let the walker go to another node following the personalization vector v with probability $1 - \alpha$.
- *Node reversal Preferential PageRank*: we apply the transformation $\mathbf{P} \rightarrow \mathbf{P} + diag(c_i)(\beta \mathbf{P}^{T_{stoch}} + (1 - \beta)\mathbf{Id})$ (the $\cdot^{T_{stoch}}$ operator is defined in 5.3), with β a constant. This amounts for the walker to jump back to a node that points towards the dangling node with a probability $\alpha\beta$, to stay on the dangling node with probability $\alpha(1 - \beta)$ or to leave the node following the teleportation vector with probability $(1 - \alpha)$.

These dangling node removal methods preserve the sparsity of the input matrix. The first method increases the score of the dangling nodes in comparison to the *Strongly preferential PageRank* method, the second one increases the score of the nodes linked to the dangling nodes. It must be emphasized that these methods do not seem to modify significantly the global PageRank score since in most applications dangling nodes represent a very small proportion of the nodes of the graph (often bogus nodes [1]).

3.1.2 Resources

We assume that we have access to an efficient method to prepare the quantum state $|v\rangle = \sum \frac{v_i}{\|v\|_2} |i\rangle$ and the oracle whose matrix representation is $(\mathbf{Id} - \alpha \mathbf{P})^T$.

3.1.3 Outputs

Our algorithm outputs a final quantum state that represents the limit probability distribution of the Google matrix in the form $|\pi\rangle = \sum_i \frac{\pi_i}{\|\pi\|_2} |i\rangle$.

3.2 Introduction to HHL

The quantum algorithm for linear systems of equations was introduced by A. Harrow, A. Hassidim and S. Lloyd in 2009 in their paper [8].

Given a linear system $\mathbf{Ax} = \mathbf{b}$ where $\mathbf{A}$ is a $n \times n$ s-sparse Hermitian matrix. The HHL algorithm prepares the solution $\mathbf{x}$ in a quantum state $|x\rangle$ using $\mathcal{O}(\frac{\log(N)s^2\kappa^2}{\epsilon})$ quantum gates, where κ is the conditioning number of $\mathbf{A}$.

For the case where the matrix $\mathbf{A}$ is not hermitian (the case for PageRank), one can solve the extended linear system $\begin{pmatrix} 0 & \mathbf{A} \\ \mathbf{A}^T & 0 \end{pmatrix} \hat{\mathbf{x}} = \hat{\mathbf{b}}$, where $\hat{\mathbf{b}} \triangleq \begin{pmatrix} \mathbf{b} \\ 0 \end{pmatrix}$ and $\hat{\mathbf{x}} \triangleq \begin{pmatrix} 0 \\ \mathbf{x} \end{pmatrix}$. The computational complexity remains the same as the eigenvalues of $\mathbf{C} \triangleq \begin{pmatrix} 0 & \mathbf{A} \\ \mathbf{A}^T & 0 \end{pmatrix}$ are the same as those of $\mathbf{A}$, each eigenvalue of $\mathbf{A}$ appearing twice in the eigendecomposition of $\mathbf{C}$.

This algorithm has important applications in several fields of Physics and Computer Science such as Electromagnetism [23] or Quantum Machine Learning [24].

Application to the PageRank algorithm

Interestingly the PageRank formalism is surprisingly well adapted to the HHL algorithm.

Let $\boldsymbol{\pi}$ be the limit probability distribution (row) vector and let $\mathbf{j}$ be the column vector with only ones, and $\mathbf{v}$ the personalization vector. Then:

$$\boldsymbol{\pi}\mathbf{G} = \boldsymbol{\pi} \tag{1}$$

$$\Leftrightarrow \boldsymbol{\pi}(\alpha\mathbf{P} + (1-\alpha)\mathbf{j}\mathbf{v}^T) = \boldsymbol{\pi} \tag{2}$$

$$\Leftrightarrow \boldsymbol{\pi}(\mathbf{Id} - \alpha\mathbf{P}) = (\alpha - 1)\boldsymbol{\pi}\mathbf{j}\mathbf{v}^T \tag{3}$$

$$\Leftrightarrow (\mathbf{Id} - \alpha\mathbf{P})^T\boldsymbol{\pi}^T = (\alpha - 1)\mathbf{v}. \tag{4}$$

Equation (4) corresponds to the linear system PageRank formulation. This formulation is equivalent to the popular eigensystem formulation used in classical approaches [3, 17]. In particular, if we write $\kappa_\infty(PageRank)$ for the condition number of $(\mathbf{Id} - \alpha\mathbf{P})$, Equation (4) presents the following property relevant for the HHL algorithm:

Property 3.1 ([17]) *The condition number $\kappa_\infty(PageRank)$ of $(\boldsymbol{Id} - \alpha\boldsymbol{P})$ is entirely determined by α:*

$$\kappa_\infty(PageRank) = \frac{1+\alpha}{1-\alpha} \tag{5}$$

Note that the row's sum of the matrix $\boldsymbol{A} = (\mathbf{Id} - \alpha\mathbf{P})$ is $(1-\alpha)$. The maximum eigenvalue of $(\mathbf{Id} - \alpha\mathbf{P})$ is then $(1-\alpha)$ (consequence of the Gershgoring theorem applied to stochastic matrices). As the eigenvalues of $\mathbf{C} = \begin{pmatrix} 0 & \mathbf{A} \\ \mathbf{A}^T & 0 \end{pmatrix}$ are the same as those of $\mathbf{A}$, and the eigenvalues and singular values of $\mathbf{C}$ coincide, the minimum singular value of $\mathbf{C}$ is $\sigma_{min} = \frac{(1-\alpha)^2}{(1+\alpha)}$.

The quantum circuit used for the HHL algorithm is sketched in Fig. 1: it uses a combination of Quantum Phase Estimation (QPE) subroutines and an oracle for real inversion. In our case, the quantum phase estimation will work on the matrix $\boldsymbol{C}$. A complete description of the steps of the HHL algorithm can be found in [8, 25] and on tutorials for quantum programming languages [26].

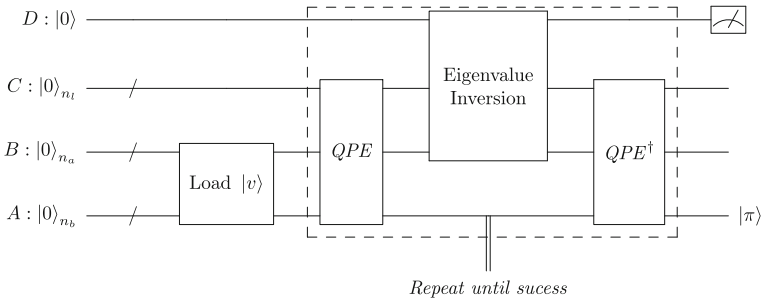


Fig. 1 Quantum circuit for the HHL algorithm

To apply the HHL algorithm, one needs to rescale the matrix $(\mathbf{Id} - \alpha\mathbf{P})$ by the $(\frac{1}{1-\alpha})$ factor so that every singular value of $\mathbf{C}$ lies between $\frac{1}{\kappa}$ and 1. We can then state the theoretical complexity for our PageRank vector computation:

Proposition 3.2 (Gate complexity of the Circuit Based Quantum PageRank) *Providing an efficient oracle access to the matrix* $\frac{1}{1-\alpha}(\boldsymbol{Id} - \alpha\boldsymbol{P})^T$*, of size* $N \times N$ *and sparsity* s*, and the personalized vector* $\boldsymbol{v}$*, the HHL-based quantum PageRank algorithm can prepare the state* $|\pi\rangle$ *with precision* ϵ*, in the runtime* $\mathcal{O}(\frac{\log(N)s^2}{\epsilon})$. □

3.3 Discussion

The use cases developed by Garnerone et al. [9] for their Adiabatic Quantum Computing method—ranking the top nodes and comparing successive PageRanks—can be analyzed with our quantum circuit. However, compared to the method in [9], which scales polylogarithmically with N and in $(\frac{1}{\epsilon^2})$, our method has a time complexity of $\mathcal{O}(\frac{\log(n)}{\epsilon})$ for a $\mathcal{O}(1)$ sparse input graph (which is often the case for web graphs)—providing efficient quantum state generation and quantum data access.

Using our method, followed by quantum (interactive) amplitude estimation, one can determine the amplitude of a given component of $|\pi\rangle$ with precision ϵ, [27, 28], which is quadratically faster than the classical MCMC algorithm.

4 Quantum algorithm for Graph Fourier Transform

In this section, we present an application of the Quantum Singular Value Transformation (QSVT) to Graph Fourier Transform. This is the key to the generalized PageRank Algorithm presented in Sect. 5.

We make use of Spectral Graph Theory (SGT) for directed graphs using Markov Chains, in a similar way as proposed recently in [7]. Previous approaches in [29, 30] yielded interesting yet less exhaustive extensions as they introduce either non-closed form expressions for *Graph Fourier Transform* [30], or do not well include approaches on undirected graphs [29]. Furthermore, the approach in [7] is interesting because it

incorporates the Random Walk framework that was already used in our subsection 2.1, and nicely defines the Graph Fourier Transform within this context.

The important notions for SGT are *Lazy Markov Chains*, *Time-Reversed Markov Chain*, *Reversible Markov chain*, *additive reversibilization* and *random walk Laplacian*. We refer the reader to [7] for their definitions.

4.1 A new approach of Graph Fourier Transform

Given a graph $G = (V, E)$, in this section and the following ones, we will frequently use the term *signal* to designate a functional $\phi : V \to \mathbb{C}$, this notion of *graph signal* is central to [7]. We define the notion of directed graph Fourier Transform as follows.

Definition 4.1 (*Directed Graph Fourier Transform*) Consider a Markov chain C on a graph $G = (V, E)$ with a probability transition matrix $\mathbf{P}$. Define $\hat{\mathbf{P}}_\alpha \triangleq \frac{\alpha\mathbf{P}+(1-\alpha)\mathbf{P}^*}{2}$, $\alpha \in [0, 1]$, where $\mathbf{P}^*$ is the transition matrix associated with the time reversed version of C. We note $\hat{\mathbf{P}}$ for $\hat{\mathbf{P}}_{1/2}$. Assume that $\mathbf{P}$ is aperiodic (otherwise one can use $\bar{\mathbf{P}} = \frac{\mathbf{Id}+\mathbf{P}}{2}$). The matrix $\hat{\mathbf{P}}_\alpha$ admits a Singular Value Decomposition: its SVD basis is called the Fourier Basis of C, denoted here with $\boldsymbol{\Xi}_{\boldsymbol{\alpha}}$. Please note that the SVD and the eigendecomposition coincide for $\alpha = 1/2$. We write $\boldsymbol{\Xi}$ for $\boldsymbol{\Xi}_{1/2}$.

Let ϕ be a *signal* on G, that is, as defined in [7], a functional $\phi : V \to \mathbb{C}$. The α-Fourier Transform of ϕ, noted $\hat{\phi}_\alpha$ is then defined by:

$$\hat{\phi}_\alpha \triangleq \boldsymbol{\Xi}_{\boldsymbol{\alpha}}{}^{-1}\phi. \tag{6}$$

In the rest of the paper, we are especially interested in the inverse Fourier Transform:

$$\phi = \boldsymbol{\Xi}_{\boldsymbol{\alpha}}\hat{\phi}_\alpha. \tag{7}$$

Our definition of graph Fourier Transform is slightly different from the one in [7]. Indeed, we do not assume $\mathbf{P}$ to be diagonalizable. We have chosen to provide a more flexible extension of this definition to non-diagonalizable transition matrices by using the SVD decomposition.

The $\alpha = 1/2$ case is particularly interesting. Indeed, if $\boldsymbol{\pi}$ is the limit distribution of C, let $\langle -, - \rangle_{\boldsymbol{\pi}}$ denote the scalar product on $\ell^2(V, \pi)$ defined as

$$\langle f, g \rangle_{\boldsymbol{\pi}} \triangleq \sum_{v \in V} f^*(v) g(v) \pi(v).$$

Since $\hat{\mathbf{P}} = \frac{\mathbf{P}+\mathbf{P}^*}{2}$ is self adjoint for the operation $\langle -, - \rangle_{\boldsymbol{\pi}}$, it is a diagonalizable operator and admits a $\langle -, - \rangle_{\boldsymbol{\pi}}$ orthonormal basis of eigenvectors $\boldsymbol{\Xi}_{1/2}$ for $\hat{\mathbf{P}}$ which is coincides with the SVD of $\hat{\mathbf{P}}_{1/2}$.

This basis is also an eigenvector basis of $\boldsymbol{\mathcal{L}}^{rw}$—the random walk Laplacian of G—which elegantly draws a parallel with the classical Fourier analysis, as well as the SGT framework for directed graphs [7].

4.2 Notion of graph Kernel

Given a Graph Fourier Transform Basis $\hat{\boldsymbol{P}}_\alpha$, a graph Kernel [4] is a vector defined directly in the Graph Fourier space.

One important example of Graph Kernel is the signal $\hat{\boldsymbol{\phi}}_{\boldsymbol{t}} = (e^{-\lambda_1 t}, e^{-\lambda_2 t}, \ldots, e^{-\lambda_n t})$, $t \in \mathbb{R}^+$, where λ_i are the eigenvalues of $\hat{\boldsymbol{P}}_\alpha$. The signal $\hat{\boldsymbol{\phi}}_{\boldsymbol{t}}$ is called the *heat Kernel* in direct reference to the Heat equation. In the rest of the paper, we designate *heat Kernels* with the letter $\mathbb{H}$.

Given a graph $G = (V, E)$, a direct transposition of the Heat equation to the directed graph setting is

$$\frac{\partial \boldsymbol{\phi}_{\boldsymbol{t}}}{\partial t} = -\boldsymbol{\mathcal{L}}^{\boldsymbol{rw}} \boldsymbol{\phi}_{\boldsymbol{t}}.$$

Let $\boldsymbol{\Xi}_{\boldsymbol{\alpha}}$ be a Fourier Basis of G; a solution of this equation, given an initial condition $\boldsymbol{\phi}_0$ and $t \in \mathbb{R}^+$ is given by

$$\boldsymbol{\phi}_t(i) = \left(\boldsymbol{\Xi}_{\boldsymbol{\alpha}} \left(e^{-\lambda_1 t}, e^{-\lambda_2 t}, \ldots, e^{-\lambda_n t} \right)^T \right)_i (\boldsymbol{\phi_0})_i .$$

The solution to the Heat equation is the reverse Fourier Transform of the Heat Kernel multiplied by an initial condition.

4.3 Inverse quantum Graph Fourier Transform: Graph Fourier eigenbasis filtering

In this section, we introduce the Inverse Quantum Graph Fourier Transform, a quantum algorithm for signal graph filtering based on the Quantum Singular Value Transformation presented by Gilyén et al. [10]. The generalized PageRank theory introduced in Sect. 5 is derived as a particular application of this algorithm. As usual, we assume given a directed weighed graph $\mathcal{G} = (V, E, \mathbf{W})$ and its associated transition matrix $\mathbf{P}$.

4.3.1 Statement of the problem

We propose in Sect. 4.3.4 an algorithm called *Inverse Quantum Graph Fourier Transform*, able to solve the following problem.

Input. Let C be a regular Markov Chain on the Graph $\mathcal{G}$ with transition matrix $\mathbf{P}$ and let $(u_1, \ldots, u_n)$ be its Fourier Basis associated with eigenvalues $(\lambda_1, \ldots, \lambda_n)$. Let $h : \mathbb{R} \to \mathbb{R}$ be a real sequentially continuous function. We assume that:

- For every $\epsilon > 0$, h admits an ϵ-close polynomial approximation by a polynomial Q of degree $d(\epsilon)$, such that $\forall x \in [-1, 1]$, $\|Q(x)\| < 1/2$. Let $h(\lambda_i)_{i\in[1,n]}$ a graph Kernel defined by $h(\lambda_i)_{i\in[1,n]} = (h(\lambda_1), \ldots, h(\lambda_n))$.
- We also assume that $\mathbf{P}$ is a regular and reversible transition matrix

Output. Given an $\epsilon \in \mathbb{R}_+$, the algorithm outputs a quantum state $|\psi_o\rangle$ representing the inverse graph Fourier transform of $h(\lambda_i)_{i\in[1,n]}$, as defined in Def. 4.1 :

$$|\psi_o\rangle = \kappa \sum_{i\in[1,n]} \tilde{h}(\lambda_i)|u_i\rangle + |o_\epsilon\rangle \tag{8}$$

κ is a constant whose value is independent from ϵ and where $\|\tilde{h} - h\|_{[0,1]} < \epsilon$ and $\||o_\epsilon\rangle\| < \epsilon$.

4.3.2 Preliminaries

In the following, three definitions are going to be heavily needed: the notion of *projected unitary encoding*, and the more specialized notions of *block encoding* and *alternating phase modulation sequence*.

Let $\widetilde{\boldsymbol{R}}$ and $\boldsymbol{R}$ be orthogonal projectors, $\mathbf{U}$ a unitary and $\mathbf{A}$ any real operator, we say that $(\widetilde{\boldsymbol{R}}, \boldsymbol{R}, \mathbf{U})$ forms a *projected unitary encoding of* $\mathbf{A}$ whenever $\mathbf{A} = \widetilde{\boldsymbol{R}}\mathbf{U}\boldsymbol{R}$. Provided that $\mathbf{A}$ is a s-qubit operator, a *block encoding of* $\mathbf{A}$ is a unitary $(\langle 0|^{\otimes a} \otimes \boldsymbol{Id}, |0\rangle^{\otimes a} \otimes \boldsymbol{Id}, \mathbf{U})$, for $a \leq s$: the matrix $\mathbf{A}$ is the top-left block of the unitary $\mathbf{U}$.

We finally define an approximate version of block encodings, as follows. Suppose $\mathbf{A}$ is a s-qubit operator, $\alpha, \epsilon \in \boldsymbol{R}_+$ and $a \in \mathbb{N}$. We say that the $(s + a)$-qubit unitary $\mathbf{U}$ is a (α, a, ϵ)-*block encoding of* $\mathbf{A}$ if

$$\|\mathbf{A} - \alpha(\langle 0|^{\otimes a} \otimes \mathbf{Id})\mathbf{U}(|0\rangle^{\otimes a} \otimes \mathbf{Id})\| \leq \epsilon. \tag{9}$$

In other words, the top left block of the matrix U is an ϵ/α-close approximation of the matrix $\mathbf{A}/\alpha$.

Projected unitary encodings are used to bridge quantum unitary operators and classical operators. Indeed, consider $(\widetilde{\boldsymbol{R}}, \boldsymbol{R}, \mathbf{U})$ a projected unitary encoding of $\mathbf{A}$. Given a degree-d odd polynomial P bounded by the absolute value 1 on the interval $[-1, 1]$, the P-singular value transformation performs the operation

$$\mathbf{A} = \widetilde{\boldsymbol{R}}\mathbf{U}\boldsymbol{R} \longmapsto P^{SV}(\mathbf{A}) = \widetilde{\boldsymbol{R}}\mathbf{U}_\Phi\boldsymbol{R} \tag{10}$$

where the unitary $\mathbf{U}_\Phi$, called the *alternating phase modulation sequence* [10] can be implemented with a circuit using $\mathbf{U}$ and its inverse d times. In the case where P is even, the same result holds, replacing $\widetilde{\boldsymbol{R}}$ by $\boldsymbol{R}$.

The circuit corresponding to the alternating phase modulation sequence $\mathbf{U}_\Phi$ is built from a vector $\Phi \in \mathbb{R}^d$ of real parameters as follows [10]:

$$\mathbf{U}_\Phi \triangleq \begin{cases} e^{i\Phi_1(2\widetilde{\boldsymbol{R}}-I)}\mathbf{U}\prod_{j=1}^{(d-1)/2}(e^{i\Phi_{2j}(2\boldsymbol{R}-I)}\mathbf{U}^\dagger e^{i\Phi_{2j+1}(2\widetilde{\boldsymbol{R}}-I)}\mathbf{U}) & \text{if d is odd} \\ \prod_{j=1}^{d/2}(e^{i\Phi_{2j-1}(2\boldsymbol{R}-I)}\mathbf{U}^\dagger e^{i\Phi_{2j}(2\widetilde{\boldsymbol{R}}-I)}\mathbf{U}) & \text{if d is even} \end{cases}$$

This unitary $\mathbf{U}_\Phi$ is a $(\widetilde{\boldsymbol{R}'}, \boldsymbol{R}', \mathbf{U}_\Phi)$ projected unitary encoding of $P^{SV}(\mathbf{A})$.

Using this notion of *alternating phase modulation sequence*, the authors in [10] present a methodology for compositionally building the vector Φ describing $\mathbf{U}_\Phi$ from the polynomial P. In the case where $\|P(x)\| \leq 1/2$, the following result holds.

Theorem 4.2 (Polynomial eigenvalue transformation [10]) *Suppose that U is an (α, a, ϵ)-encoding of a Hermitian matrix A. Moreover, let $\delta \geq 0$ and $P_{\mathbb{R}} \in \mathbb{R}[x]$ is a degree-d polynomial satisfying that $\forall x \in [-1, 1] : |P_{\mathbb{R}(x)}| \leq \frac{1}{2}$.*

Then there is a quantum circuit $\tilde{U}$, which is an $(1, a + 2, 4d\sqrt{\epsilon/\alpha} + \delta)$-encoding of $P_{\mathbb{R}}(A/\alpha)$, and consists of d applications of U and $U^\dagger$ gates, a single application of controlled-U and $\mathcal{O}((a + 1)d)$ other one- and two-qubit gates. Moreover, we can compute a description of such a circuit with a classical computer in time $\mathcal{O}(poly(d, \log(1/\delta)))$.

4.3.3 Set-up for the algorithm

Instead of directly working with $\mathbf{P}$, the algorithm relies on $\boldsymbol{\mathcal{P}}$ defined as $\boldsymbol{\Pi}^{1/2}\mathbf{P}\boldsymbol{\Pi}^{-1/2}$, where $\boldsymbol{\Pi}$ is the probability limit distribution matrix of $\mathbf{P}$: It is defined as: $\boldsymbol{\Pi} := diag(\boldsymbol{\pi}) = diag(\pi_1, \ldots, \pi_N)$.

We then have $\boldsymbol{\Pi}^{\pm 1/2} = diag(\pi_1^{\pm 1/2}, \ldots, \pi_n^{\pm 1/2})$, well-defined because for all i, $\pi_i > 0$. The form $\boldsymbol{\mathcal{P}} = \boldsymbol{\Pi}^{1/2}\mathbf{P}\boldsymbol{\Pi}^{-1/2}$ yields interesting properties. Indeed, $\boldsymbol{\mathcal{P}}$ has the same spectrum as $\mathbf{P}$ and is self adjoint for the scalar product $\langle -, - \rangle_{\boldsymbol{\pi}}$. This makes its eigenvalues coincide with its singular values. The matrix $\boldsymbol{\mathcal{P}}$ has then an $(1, a, 0)$ unitary encoding that can be explicitly built out of classical random walk operators (see Sect. 4.3.6). This makes it possible to build on the assumptions of [10, 31, 32], and rely on their results.

This algorithm relies on the following property: for normal semi-definite matrices the eigendecomposition and SVD coincide. Hence, reversible transition matrices $\mathbf{P}$ being adjoint operators in the space $l^2(V, \pi)$, the operator $\boldsymbol{\mathcal{P}} = \boldsymbol{\Pi}^{1/2}P\boldsymbol{\Pi}^{-1/2}$ is an adjoint operator in the classical $l^2(V)$ space, equipped with the standard inner product—i.e., $\boldsymbol{\mathcal{P}}^* = \boldsymbol{\mathcal{P}}^T = \boldsymbol{\mathcal{P}}$. The whole framework developed in Section 4.1 then directly applies to the SVD of $\boldsymbol{\mathcal{P}}$, which explains how one can simply identify the singular values to eigenvalues and eigenvectors to singular vectors for the matrix $\boldsymbol{\mathcal{P}}$.

Our algorithm also applies to non-reversible transition matrices; however, one loses some of the properties introduced in Sect. 4.1: we discuss the non-reversible case in Sect. 4.4.

4.3.4 A high level description of the Inverse Quantum Graph Fourier Transform

Figure 2 presents a quantum circuit implementation of the Inverse Quantum Graph Fourier Transform. It consists in 3 main steps, sketched below and described in more details in the remainder of the section.

Step 1. We suppose being given $2b = 2\log_2(N)$ initial qubits in the state $|0\rangle_A|0\rangle_B$, in this way the register A and the register B contain both exactly b qubits. Using an SVD on $\boldsymbol{\mathcal{P}}$, prepare the initial quantum state $|\psi_P\rangle = \sum \mathcal{P}_{i,j}|i\rangle_A|j\rangle_B = \sum_{k=1}^{N} \sigma_k|v_k\rangle_A|v_k\rangle_B$, in the same way as proposed in [33] and [34], where $|v_k\rangle$ is the quantum state representing the $k$ith singular vector (or eigenvector) of $\mathcal{P}$.

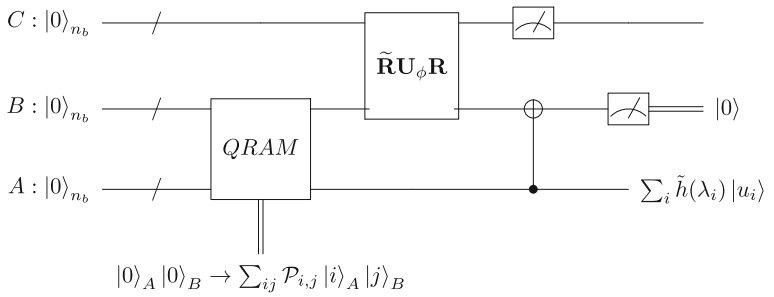


Fig. 2 The Quantum Circuit used by the Graph Fourier eigenbasis filtering

The preparation of this initial state can be performed by addressing a QRAM where one can retrieve the values of $\mathcal{P}_{i,j}$ for state preparation. Section 4.3.5 discusses the QRAM construction.

Step 2. This step consists in combining the results of [10] to build a circuit that can produce a succession of well-chosen singular value polynomial transformations. We start by building two unitaries that apply the polynomial transformations

- $A \rightarrow S^{(SV)}(A)$, where S is a polynomial approximation of the function $x \rightarrow 1/x$ over $[-1, 1]$, and
- $A \rightarrow Q^{(SV)}(A)$, where Q is an ϵ-close polynomial approximation of h.

Then, one can use the block matrix multiplication results presented in Lemma 53 of [10] to build a final block unitary encoding $\widetilde{\boldsymbol{R}}\boldsymbol{U}_{\boldsymbol{\phi}}\boldsymbol{R}$ representing the operation $A \rightarrow (S \times Q)^{(SV)}(A)$.

Section 4.3.6 provides a detailed explanation of the various constructions.

Step 3. Apply the circuit realizing $\widetilde{\boldsymbol{R}}\boldsymbol{U}_{\boldsymbol{\phi}}\mathbf{R}$ to the register B. The state of the system is then $|\psi_3\rangle = \sum_i Q(\lambda_i)|v_i\rangle|v_i\rangle$.

Finally, uncompute the register B. This can be done without the need of external ancilla qubits, by only applying CNOT gates to every pair of qubits from register A and B. After the operation, the register B will be in the pure quantum state $|0\rangle_B$ and one can uncompute it by measurement.

Accounting for the approximations, the state of the system is at the end of the algorithm $|\psi_f\rangle = \kappa \sum_i Q(\lambda_i)|v_i\rangle + |o_\epsilon\rangle$.

4.3.5 Step 1: Discussion for the need of a QRAM

All these methods suppose to possess an efficient access to a quantum memory (QRAM for instance) being able to represent classical data in a quantum state. The paper [35] introduced a Bucket Brigade QRAM structure which can provide and store classical information in $\mathcal{O}(poly - log(n))$ complexity. However, this structure does not seem to provide a sufficient stability to guarantee constant error prediction for sub-linear complexity quantum based algorithms (see [36] for more detailed explanations). Other data structures were introduced since the Bucket Brigade version, see [37] for instance.

One must emphasize that the complexity results depend heavily on the regularity of the function h. Indeed, the algorithm time complexity depends directly on the

degree of a polynomial approximation of h. We will propose a detailed analysis of the complexity of our quantum algorithm in Sect. 4.3.8.

Unitary encoding of the transition matrix.

In our SVT algorithm, we build explicitly a unitary encoding $(\widetilde{\boldsymbol{R_0}}, \boldsymbol{R_0}, \mathbf{U})$ of $\mathcal{P}$, inspired from the construction proposed in [10] in the *Corollary 34*, which stems from Szegedy's construction of Quantum Walk Operators (see [38]). Indeed, the unitary $\mathbf{U} = \mathbf{U}_L^\dagger \mathbf{U}_R$ is built from the two state preparation unitaries $\mathbf{U}_L$ and $\mathbf{U}_R$:

$$\mathbf{U}_R : |0\rangle|x\rangle \rightarrow \sum_{y \in V} \sqrt{P_{xy}}|x\rangle|y\rangle \tag{11}$$

$$\mathbf{U}_L : |0\rangle|y\rangle \rightarrow \sum_{x \in V} \sqrt{P^*_{yx}}|x\rangle|y\rangle \tag{12}$$

This implies, choosing $\boldsymbol{R_0} = \widetilde{\boldsymbol{R_0}} = (|0\rangle\langle 0| \otimes \mathbf{Id})$, that $\widetilde{\boldsymbol{R_0}} \boldsymbol{U} \boldsymbol{R_0} = \mathcal{P}$.
We remind that $P^*_{x,y} := P_{y,x} \frac{\pi(y)}{\pi(x)}$, which allows getting $\mathcal{P}$ from $(\sqrt{P_{xy} P^*_{yx}})_{ij \in [1,n]}$.

4.3.6 Step 2: Building the approximation unitaries

If $\boldsymbol{U}$ is a $(1, a, \epsilon)$ block encoding of an operator $\boldsymbol{A}$, such that $(\boldsymbol{R}, \widetilde{\boldsymbol{R}}, \boldsymbol{U})$ form a projected unitary encoding of $\boldsymbol{A}$, we will use the more natural notation $\boldsymbol{U} \overset{\epsilon}{\simeq} \begin{pmatrix} \mathcal{P} & 0 \\ 0 & * \end{pmatrix}$ to imply the existence of a projected unitary encoding such that:

$$\|\mathcal{P} - \widetilde{\boldsymbol{R}} \boldsymbol{U} \boldsymbol{R}\| \leq \epsilon$$

First unitary: inverse function approximation. To build a unitary that approximates the inverse function, we are going to use the following result from [10] (note that we are changing the constant factor from 3/4 to 1/2 so that we can use Theorem 4.2):

Corollary 4.3 (Approximation of $1/x$, [10]) *Let $\epsilon, \delta \in (0, \frac{1}{2}]$, then there is an odd polynomial $S \in \mathbb{R}[x]$ of degree $\mathcal{O}(\frac{1}{\delta} \log(\frac{1}{\epsilon}))$ that is ϵ-approximating $f(x) = \frac{1\delta}{2x}$ on the domain $I = [-1, 1] - [-\delta, \delta]$, moreover it is bounded by 1/2 in absolute value.*

Since the matrix $\mathcal{P}$ is non-singular, there exist a $\delta > 0$ such that $\forall x \in [-\delta, \delta], x \notin Sp(\mathcal{P})$ where $Sp(\mathcal{P})$ is the spectrum of $\mathcal{P}$. Then, according to the corollary above, there is an odd polynomial $S \in \mathcal{R}[X]$ of degree $\mathcal{O}(1/\delta \log(1/\epsilon))$ that is $\epsilon - approximating$ the function $f(x) = \frac{1\delta}{2x}$ on the domain $I = [-1, 1] \backslash [-\delta, \delta]$. [10] gives a method to build recursively the polynomial S using bounded polynomial approximation of local Taylor series—explaining the process that led to such a construction is out of the scope of this paper.

More precisely, by choosing $\delta = min_{i \in [|1,N|]} \{\lambda_i \in Sp(\mathcal{P}), \|\lambda_i\|\}$, the conditions for 4.2 and 4.3 are satisfied, and there exists U_S being an $(1, a+2, \epsilon_1)$ unitary encoding of $S(\mathcal{P})$, where $\epsilon_1 \in \mathbb{R}_+$ can be chosen arbitrarily small (the gate complexity to build the circuit evolves as $poly(log(1/\epsilon_1))$). Then one can build the operator U_S such that

(with a precision ϵ_1)

$$\mathbf{U}_S \overset{\epsilon_1}{\simeq} \begin{pmatrix} S^{(SV)}(\mathcal{P}) & 0 \\ 0 & * \end{pmatrix}$$

Second unitary: h **function approximation**. Since we have assumed that Q is an ϵ approximation of h, bounded in norm by $1/2$, we can directly use here Theorem 4.2 to build the operator $\mathbf{U}_Q$, being a $(1, a+2, \epsilon_2)$ unitary encoding of $Q(\mathcal{P})$ - ϵ_2 can be chosen arbitrarily small, the complexity of the circuit evolving as $poly(log(1/\epsilon_1))$, i.e., $\boldsymbol{U_Q}$ is a unitary that obeys the following equation (with a precision ϵ_2):

$$\mathbf{U}_Q \overset{\epsilon_2}{\simeq} \begin{pmatrix} Q^{(SV)}(\mathcal{P}) & 0 \\ 0 & * \end{pmatrix}.$$

Patching up: multiplication of the previous encodings We use the following multiplication theorem for block-encoded unitaries:

Theorem 4.4 (Product of block-encoded matrices [10]) *If U is an (α, a, δ)-block-encoding of an s-qubit operator A and V is an (β, b, ϵ) encoding of an s-qubit operator B, then $(I_b \otimes U)(I_a \otimes V)$ is an $(\alpha\beta, a+b, \alpha\epsilon + \beta\delta)$-block-encoding of AB*

This theorem provides a final $(1, 2a+4, \epsilon_1 + \epsilon_2)$ unitary encoding of $(S \times Q)^{(SV)}(\mathcal{P})$: $\boldsymbol{U_\phi}$. In other words, we have:

$$\boldsymbol{U_\phi} \overset{\epsilon_1+\epsilon_2}{\simeq} \begin{pmatrix} (S \times Q)^{(SV)}(\mathcal{P}) & 0 \\ 0 & * \end{pmatrix}$$

4.3.7 Step 3: Derivation of the output state

In the step 3, applying $\widetilde{\boldsymbol{R}}\boldsymbol{U_\phi}\boldsymbol{R}$ to the register B and a b-size ancilla set of qubits, yields:

$$\sum_{k\in[1,n]} \sigma_k |v_k\rangle_A \widetilde{\mathbf{R}}\mathbf{U}_\phi \mathbf{R}(|v_k\rangle_B |0\rangle_C) \tag{13}$$

$$= \sum_{k\in[|1,n|]} \sigma_k |v_k\rangle_A \left(\sum_{i\in[|1,n|]} SQ(\sigma_i)|v_i\rangle\langle v_i||v_k\rangle_B |\zeta\rangle_C \right) \tag{14}$$

$$= \sum_{k\in[|1,n|]} \sigma_k \left(\kappa \frac{Q(\sigma_k)}{\sigma_k} + \epsilon_k \right) |v_k\rangle_A |v_k\rangle_B |\zeta\rangle_C \tag{15}$$

$$= \sum_{k\in[|1,n|]} \kappa\, Q(\sigma_k)|v_k\rangle_A |v_k\rangle_B |\zeta\rangle_C + |o_\epsilon\rangle \tag{16}$$

$$= \sum_{k\in[|1,n|]} \kappa\, Q(\sigma_k)|v_k\rangle_A |v_k\rangle_B + |o_\epsilon\rangle \tag{17}$$

The value κ is a constant whose origin is explained in more detail in Sect. 4.3.6. We define $|o_\epsilon\rangle = \sum_{k\in[|1,n|]} \sigma_k \epsilon_k |v_k\rangle_A |v_k\rangle |\zeta\rangle_C$. Here, $\||o_\epsilon\rangle\| \leq \epsilon$. We saw in Sect. 4.3.6 that we can choose the algorithm parameters so that the norm of $|o_\epsilon\rangle$ can be arbitrarily small (to the expense of a greater circuit complexity). Let's simply recall that the states $|v_i\rangle$ form an orthonormal basis of vectors, which allows one to derive the second line equality in the previous set of equations.

The register C being unentangled with the register B, due to the diagonal block-form of the operator $\widetilde{\mathbf{R}}\mathbf{U}_\phi\mathbf{R}$, one can uncompute the register without modifying the global state of the system.

4.3.8 Complexity analysis

Our algorithm being the combination of three macroscopic quantum processes, one has to estimate the gate and memory cost of each process and sum each individual computational cost to get the global computational cost.

Cost of QRAM state preparation. As noticed in 4.3.5, several authors have introduced Quantum Random Access Memory structures that are able to prepare any loaded state in time $\mathcal{O}(polylog(n))$ with a low degree of polynomial scaling.

Cost of $\widetilde{\mathbf{R}}\mathbf{U}_\phi\mathbf{R}$ ***preparation:*** Let $(\widetilde{\mathbf{R}}, \mathbf{R}, \mathbf{U})$ be a unitary encoding of $\mathbf{P}$, a real operator. According to the results of [10] (*Corollary 18 & Lemma 19*), given a real odd polynomial of degree d, bounded by 1 in [0, 1], one can build $\mathbf{U}_\phi$—after $\mathcal{O}(poly(d, log(\frac{1}{\epsilon})))$ preprocessing computations on classical computer—using for d-times:

- The operators $\mathbf{U}$ and $\mathbf{U}^\dagger$
- The gates C_RNOT and $C_{\widetilde{R}}NOT$
- Single qubit gates.

the process uses a constant number of ancilla qubits. Which makes the overall number of gates required scaling as $\mathcal{O}(d\mathcal{T}(\mathbf{U}))$ with $\mathcal{T}(\mathbf{U})$ the cost of implementing C_RNOT and U gates [10].

Cost of last C_{NOT} ***preparation:*** The last C_RNOT operator requires the use of n_b 2-qubit $CNOT$s, which makes an overall of $\mathcal{O}(\log(n))$ quantum gates.

Patching up. By combining all the previous costs, one can deduce that the overall quantum gate cost for the circuit implementation 2 scales as $\mathcal{O}(d\mathcal{T}(U) + S(\log(n)))$ with S a low degree polynomial. This result proves a direct dependency between the number of quantum gates required and the degree of the simulated polynomial: this explains why our algorithm is most efficient because, in general, Fourier Kernels are well approximated by low degree polynomials.

4.4 Discussion

Even though we have only worked with directed graphs, our Quantum Graph Frequency-Based Inverse Fourier Transform transposes well to the undirected framework. Indeed, undirected graphs are always reversible and symmetric which means that the reversible condition on the input may be dropped in the undirected setting.

Several Spectral Graph Theory applications only make use of undirected graphs [4, 6, 39] and our algorithm may be of great use in those cases. Please note also that these undirected graphs does not need to be regular and that our results also apply to undirected non-regular graphs (whereas power method for PageRank does not converge for these graphs).

As discussed before our algorithm is adapted for regular non-reversible graphs: however, in this case, one loses the direct correspondence between transition matrix singular values and eigenvalues due to the absence of symmetries for the transition matrix. Indeed, the operator $\mathcal{P} = \mathbf{\Pi}^{-1/2}\mathbf{P}\mathbf{\Pi}^{1/2}$ is no longer self-adjoint and one can only claim that the singular values of $\mathcal{P}$ are the square root of the eigenvalues of $\mathcal{P}\mathcal{P}^* = \mathbf{\Pi}^{-1/2}\boldsymbol{P}\boldsymbol{P^*}\mathbf{\Pi}^{1/2}$.

Note that $Sp(\mathbf{\Pi}^{-1/2}\boldsymbol{P}\boldsymbol{P^*}\mathbf{\Pi}^{1/2}) = Sp(\boldsymbol{P}\boldsymbol{P^*})$.

5 Spectral graph theory quantum PageRank algorithm

In this section, we introduce a generalized version of the PageRank using Spectral Graph Theory. Using results stemming from Spectral Graph Theory, we are able to derive a generalization of the PageRank algorithm that is able to better translate local graph properties on the nodes ranking produced by the algorithm. This work is the first in its kind, as—to our knowledge—a complete adaptation of the spectral graph theory to the PageRank has not yet been proposed. Our algorithm is able to adjust the influence of local and global graph structures to the final ranking and can provide a whole new class of importance criteria that can be used to produce new node rankings for any graph.

Throughout this section, we will assume given an input directed weighed graph $\mathcal{G} = (V, E, \mathbf{W}(E))$ *and its associated transition matrix* $\mathbf{P}$.

This generalized PageRank can be seen as a detailed application for the Inverse Graph Fourier Transform Quantum Algorithm of Sect. 4.

Indeed, the algorithm proposed in Sect. 4 can be directly used, in the PageRank framework discussed in subsection 5.1, as it can simulate any ϵ-polynomial approximate function of the transition matrix eigenvalues for any reversible Markov Chain.

A possible use case applied to the PageRank problem could be the following: after having computed a classical ranking with the ADSM (Almost Directed Stochastic Matrices) structure introduced in Sect. 5 in a number of operations scaling linearly with the size of the input matrix, N, one can use the limit probability distribution vector to build a reversible transition matrix (in time $\mathcal{O}(N)$ due to the sparsity of ADSM) and apply the algorithm to build a finer ranking. Then amplitude estimation routines can be applied to the final quantum state to get refined rankings for hardly distinguishable sets of nodes. Since ADSM preserves strongly connected components, our quantum PageRank algorithm may be applied directly to a given connected component to produce a final quantum state representing the nodes of this particular connected component.

5.1 Generalized importance vector

In this subsection we precise the notion of ranking by *order of importance* and generalize this notion, providing a new class of *Generalized PageRank rankings*

Definition 5.1 (*Generalized importance vector*) Let $\lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n$ be the eigenvalues of the Laplacian associated with a given transition matrix $\mathbf{P}$ of the input graph and $(u_1, \ldots, u_n)$ a basis of unit eigenvectors associated to these eigenvalues.

Given a non-increasing Graph Kernel $\mathbb{K}$ (see Sect. 4.2), the $\mathbb{K}$-generalized importance vector is the inverse Fourier Transform of $\mathbb{K}$, i.e., such that:

$$\forall (i, j) \in V^2 . i \leq j \implies \mathbb{K}_i \geq \mathbb{K}_j. \tag{18}$$

Here are some remarks regarding the previous definition:

- Most of the time, the transition matrix $\mathbf{P}$ will be chosen to be either the classical or the personalized Google Matrix. Since these matrices are regular and easily computable. However, we have also found that other candidates yield interesting properties: we will give some examples and discuss their usefulness in our framework.
- A first useful example of a generalized importance vector is given by the Kernel $\boldsymbol{\delta^0}$, where $\delta_i^0 = \delta_{i \in \mathbb{J}}$ where $\mathbb{J} = \{i \in V \text{ st: } \lambda_i = 0\}$ and λ_i is the ith eigenvalue of the Laplacian of the Google Matrix. In the case of the Google Matrix, this amounts to compute the classical PageRank, and then to compute the probability limit distribution of $\mathcal{G}$. Thus, we call the Kernel $\boldsymbol{\delta^0}$, the classical PageRank Kernel.
- An interesting choice of Kernel may be the Heat Kernel $\mathbb{H} = (e^{-\lambda_1 t}, e^{-\lambda_2 t}, \ldots, e^{-\lambda_n t})$, $t \in \mathbb{R}^+$, this Kernel models the propagation of heat waves on the Laplacian: for a given choice of t, one can choose the duration of the heat propagation, and then carefully adjust the locality of the ranking. Note that the work from [40] corresponds in fact to the Heat Kernel particular case of our PageRank generalization.
- The approach presented in [41] can also be treated in our framework. Indeed, the power series decomposition $\sum_{k \in [|0,\infty|]} \gamma_k \mathbf{P}^k$ can be rewritten, using the eigendecomposition of $\mathbf{P} = \mathbf{J}\mathbf{U}\mathbf{J}^{-1}$:

$$\sum_{k \in [|0,\infty|]} \gamma_k \mathbf{P}^k = \mathbf{J} \left(\sum_{k \in [|0,\infty|]} \gamma_k \left(diag\left(\lambda_1^k, \ldots, \lambda_n^k\right) + \mathbf{R}^k \right) \right) \mathbf{J}^{-1} \tag{19}$$

$$= J(diag(f(\lambda_1), \ldots, f(\lambda_n)) + \sum_{k \in [|0, r^{max}|]} \gamma_k Rem^k) J^{-1} \tag{20}$$

where $r^{max} < \infty$ and $f : \mathbb{C} \to \mathbb{C}; z \to \sum_{k \in [|0,\infty|]} \gamma_k z^k$ is an analytic function.

A comment on the Google Matrix

The Fourier Transform framework allows us to derive a generalized version of the PageRank algorithm which may better distinguish secondary hubs of nodes. The generalized importance vector allows one to fine-tune the balance between global graph structure and local graph properties.

In fact, this property can be easily understood for the *Heat Kernel*, $\mathcal{H}$: when the parameter t is close to 0, the heat has not yet propagated through the whole input graph, leading to a more prominent local description of the input graph; whereas when $t \to \infty$, one has access to a global description of the properties of the input graph.

To fully exploit the power of Spectral Graph Theory, we have introduced several classes of candidate matrices that can be used to compute the Generalized PageRank: indeed, we have seen in section 4.1 that only the properties of connectivity and aperiodicity are required to be able to apply the Perron–Frobenius Theorem. Thus the original Google Matrix captures a very specific case of matrix regularization, which does favor general graph structure over local properties by adding a complete graph with low transition weights to the original graph. Similarly, the personalized PageRank use of the personalization vector does refine the ranking around a specific node but does introduce dense matrix rows which may be complicated to deal with in practice [17].

5.2 Generalizing the Google matrix

We introduce a generalization of the Google Matrix and study some interesting applications of our novel PageRank Matrices—Almost Directed Stochastic Matrices (ADSM)—while proving that the classical PageRank Matrix is in fact a very particular instance of ADSM.

This generalized version of the Google Matrix heavily relies on the Spectral Graph Theory as it is inspired from the use of the *reversibilized* and *lazy* matrices versions in the transition matrix regularization process. Indeed, these two extended matrix forms are useful to prepare admissible candidates to the PageRank theorem—i.e., regular Matrices.

5.2.1 Patching the dangling nodes

We will use the following version of the Weakly Preferential PageRank method (see Sect. 2.2):

$$\mathbf{P} = \bar{\mathbf{P}} + \sum_{i \leq C} c_i \mathbf{u}_i^T \tag{21}$$

where C is the number of connected components of $\mathcal{G}$ and c_i the indicator matrix for the ith connected component and u_i a uniform distribution on the ith connected component of the graph.

This way to patch dangling nodes is interesting as it doesn't link two seemingly unrelated connected components: this is necessary if these components stay distinct in the final google matrix representation.

5.2.2 Almost directed stochastic matrices

Next, we introduce the notion of Loop Completing Graph Extension that heavily simplifies the expression of our generalized Google Matrix.

Definition 5.2 (*Loop Completing Graph Extension*) Let $G = (V, E, \mathbf{W}(E))$ be a directed weighted graph, we call a **loop completing directed weighted graph extension** (or simply loop completing graph extension) of G, $G^e = (V, E^e, \mathbf{W}(E)^e)$, a graph that has the following properties:

1. G is a subgraph of G^e, i.e., $E \subset E^e$
2. $\forall x \in V, (x, x) \in E^e$: the graph extension has self-looping edges.
3. $\forall (i, j) \in E^e \backslash E, (j, i) \in E^e$: the graph extension edges are undirected.

Then we rely on loop completing graph extensions to define a Generalized PageRank matrix—the *Almost Directed Stochastic Matrices*, or ADSM.

The intuition behind ADSM is the following: given two nodes of any graph $\mathcal{G}$, x and y, and a Markov chain C on $\mathcal{G}$, and assuming that the initial distribution probability of the walkers is uniform (i.e., $\forall i \in V, \mathbb{P}(X_0 = i) = \frac{1}{|V|}$) we have the following equalities stemming from Bayes probability formula:

$$\mathbf{P}^{T_{stoch}}(y, x) := \mathbb{P}(X_0 = y | X_1 = x) = \frac{\mathbb{P}(X_1 = x | X_0 = y)\mathbb{P}(X_0 = y)}{\sum_{k \in V} \mathbb{P}(X_1 = y | X_0 = k)\mathbb{P}(X_0 = k)} \tag{22}$$

$$= \frac{\mathbb{P}(X_1 = x | X_0 = y)}{\sum_{k \in V} \mathbb{P}(X_1 = y | X_0 = k)} \tag{23}$$

$$= \frac{P_{y,x}}{\sum_{k \leq n} P_{k,x}} \tag{24}$$

Then, one can define a *weak reversibilization* version $\mathbf{P}^{T_{stoch}}$ of a transition matrix using the last equation. This weak reversibilization unveils interesting results when walkers are uniformly distributed initially.

Equation 22 shows a locally tempered behavior for the reversed edges. The more connected the node y, the lower $\mathbb{P}(X_1 = x | X_0 = y)$ and the higher $\mathbb{P}(X_1 = y)$ will be: reversed edges penalize high connected nodes, producing a fairer ranking. Thus, a an interpretation of weak and strong reversibilization is the following: the weak reversibilization satisfies the power equilibrium principle in the immediate neighborhood of each node, whereas the strong reversibilization satisfies a global power equilibrium principle, thanks to the limit probability distribution matrix $\mathbf{\Pi}$.

These matrices are called "almost directed" because the weight attributed to the reverse artificial nodes is small enough so that the probability of the random walker getting through these nodes is quite low (even though not equal to zero):

Definition 5.3 (*Almost Directed Stochastic Matrices*) Let $\mathbf{P}$ be a left stochastic matrix. We define:

$$\mathcal{P} = (1-\alpha)\mathbf{P} + \alpha(\mathbf{G}^{gen})^{T_{stoch}} \tag{25}$$

with

$$(\mathbf{G}^{gen})^{T_{stoch}} := (\mathbf{G}^{gen})^{T} \boxtimes \frac{1}{\mathbf{G}_1^{gen}}$$

and where

$$\left(\frac{1}{\mathbf{G}_1^{gen}}\right)_i := \begin{cases} \frac{1}{\sum_{j\leq n} G_{ji}^{gen}} & \text{if } i \text{ has incoming nodes} \\ 0 & \text{otherwise} \end{cases}$$

is a column vector and $\boxtimes$ is the elementwise product (ith row of $(\mathbf{G}^{gen})^T$ multiplied by $(\frac{1}{\mathbf{G}_1^{gen}})_i$). α is a constant (usually small).

The *Almost Directed Stochastic Matrices* is *regular*, which means that one can apply the Perron–Frobenius theorem and compute a probability limit distribution.

Proof Let $\mathbf{P}$ be this stochastic matrix, then by column permutation we can find a basis where $\mathbf{P}$ has the form:

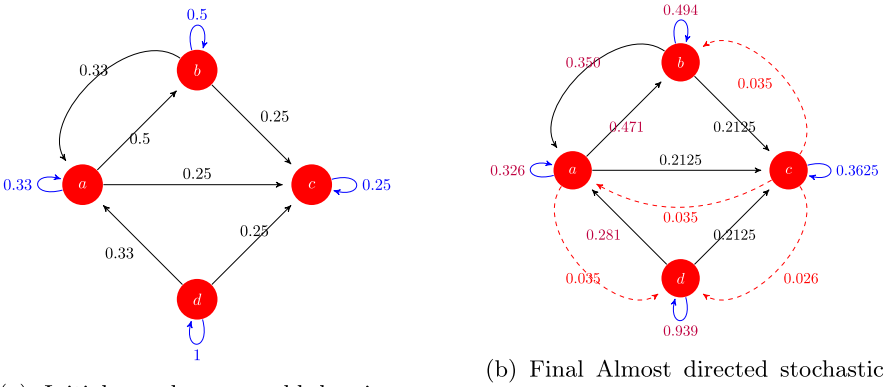
$$\mathbf{P} = \begin{pmatrix} \mathbf{P}_1 & 0 & \cdots & 0 \\ 0 & \mathbf{P}_2 & \cdots & 0 \\ & & & \\ 0 & 0 & \ddots & 0 \\ 0 & 0 & 0 & \mathbf{P}_n \end{pmatrix} \tag{26}$$

where $\mathbf{P}_i$ is a stochastic matrix representing a strongly connected directed graph. Then $\mathcal{P} = \begin{pmatrix} \mathcal{P}_1 & 0 & \cdots & 0 \\ 0 & \mathcal{P}_2 & \cdots & 0 \\ & & & \\ 0 & 0 & \ddots & 0 \\ 0 & 0 & 0 & \mathcal{P}_n \end{pmatrix}$ where $\mathcal{P}_i$ is a regular stochastic matrix. The result holds. □

We introduce the *Weak Stochastic Transposition* operator—$(\cdot)^{T_{stoch}}$—such that $(\cdot)^{T_{stoch}} : \mathbf{P} \to \mathbf{P}^{T_{stoch}}$, maps the set of left-stochastic matrices onto itself. This operator acts on any stochastic matrix (not solely the regular ones) and outputs a transition matrix with reversed edges by using only the original transition matrix. This operator is called *Weak*, because it does not bear the usual transposition properties (linearity for instance), contrary to the reversibilization operator $(\cdot)^*$ that displays a behavior similar transposition. Thus, one can regard the $(\cdot)^*$ operator as a *Strong Stochastic Transposition*, that only acts on regular left-stochastic matrices.

The *weak stochastic transition* can be efficiently constructed by saving the calculation of the sums $\frac{1}{\mathbf{G}_1^{gen}}$ in the memory and updating them instead.

Figure 3 provides examples of ADSM graph construction.



(a) Initial graph : we add looping edges to make the graph aperiodic

(b) Final Almost directed stochastic matrix : we added multiple reversed edges.

Fig. 3 A construction example for Almost Directed Stochastic Matrices

Besides, the notion of *Almost Directed Stochastic Matrices* using *Loop Completing Graph Extension* can also be seen as a generalization of the classical PageRank. In fact, the google matrix is the particular case in which $\mathcal{G}^e$ represents a complete graph.

Hence, the introduction of almost directed stochastic matrices based on the completing graph extension is relevant because according to the shape of the completing graph extension, the heat flow through the graph will change and the local properties of the graph won't be translated the same way in the final result.

5.3 Generalized PageRank

The generalized PageRank introduced in this section can then be computed by:

1. Patching the dangling nodes of the input transition matrix, P using the Weak preferential PageRank method (see Sect. 3.1).
2. Compute the almost directed stochastic matrix $\mathcal{P}$ associated with the input graph. The reversed ADSM ($\frac{\mathcal{P}+\mathcal{P}^*}{2}$) can be used to provide results that can be interpreted more easily (as the eigenvalues and the singular values are the same). For that, one needs to compute the limit probability distribution of the transition matrix and use the closed form expression $P^*_{x,y} \triangleq P_{y,x}\frac{\pi(y)}{\pi(x)}$ to compute $\mathcal{P}^*$.
3. Compute the SVD of $\mathcal{P}$
4. Compute the graph Kernel associated with the singular values of $\mathcal{P}$, using the Inverse Graph Fourier Transform Quantum Algorithm of Sect. 4.
5. Use the kernel values to rank the nodes of $\mathcal{P}$.

5.4 Results and analysis

To better grasp our PageRank algorithm extensions we have produced several graph rankings on a classical computer, following the algorithm described in 5.3. Here we present results for various input graphs. In these numerical applications, we have used

the *reversed ADSM*, which is the *reversibilized version* of the ADSM—$\frac{\mathcal{P}+\mathcal{P}^*}{2}$—to comply with the definition of the directed graph Fourier transform in 4.1.

5.4.1 Summary

In this section, we first summarize our experimental findings. Section 5.4.2 details them thoroughly.

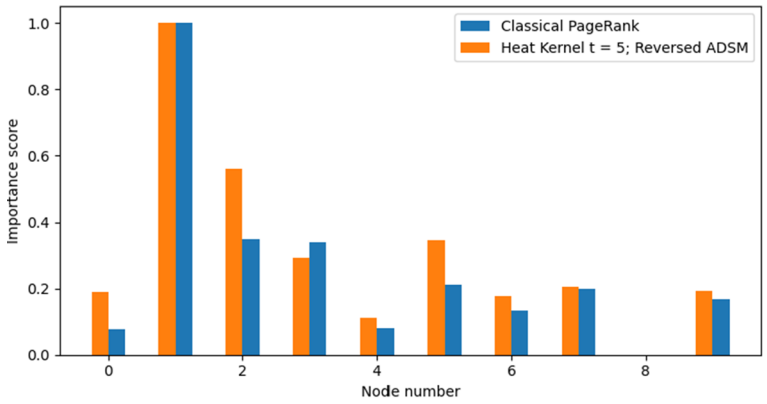
- Heat Kernels with a low parameter t give to the Quantum PageRank the same highlighting behavior as the Classical PageRank.
- On the opposite, Heat Kernels with a high parameter t provides Quantum PageRanks that highlight better the auxiliary nodes than the Classical PageRank.
- Less common rankings with external nodes appearing with a high ranking can be prepared with other kinds of kernels. One can think of them as specialized queries.

5.4.2 Complete discussion

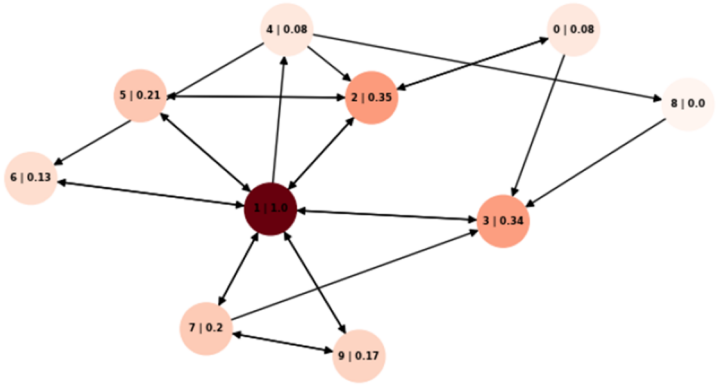
Figures 4 and 5 compare the classical PageRank method to a Heat Kernel method (parameter $t = 5$) Importance distributions. The numerical data used to build these plots are provided in Tables 1 and 2. In Fig. 6, one can note that the importance vectors produced are fairly close to the standard classical PageRank importance vectors—the greater the time, the more damped the Kernel distribution and the Importance vector gets closer to the stationary distribution. One can remark that for lower t parameters ($t = 5$ and below), the importance vector unveils secondary hubs more clearly—this is shown by the greater oscillations of the Generalized Importance Vector associated with these Kernels. These oscillations can be clearly seen in Figs. 8 and 7 where the Heat Kernel based Importance vectors attributes a higher score for nodes of secondary importance (i.e., for nodes ranked after the fifth position for these graphs) when compared to the classical PageRank.

Figure 7 shows that this effect is even more accentuated for Heat Kernels with low time parameters ($t \leq 5$). This secondary hub highlight effect appears clearly for sufficiently large graphs (50 nodes and more), as it seems that, for very small graphs several boundary effects may affect the global ranking. Yet, this secondary node enhancement can be clearly seen in Figs. 4 and 5: for Fig. 4 the effect is clear for the nodes 0, 3, 4, 5, 6 and 9; and for Fig. 5, the nodes 0, 3, 4 and 10 exhibit an enhanced importance score when compared to the classical PageRank. This results in a more balanced importance distribution for the graphs associated with the Heat Kernels in Fig. 4 and 5.

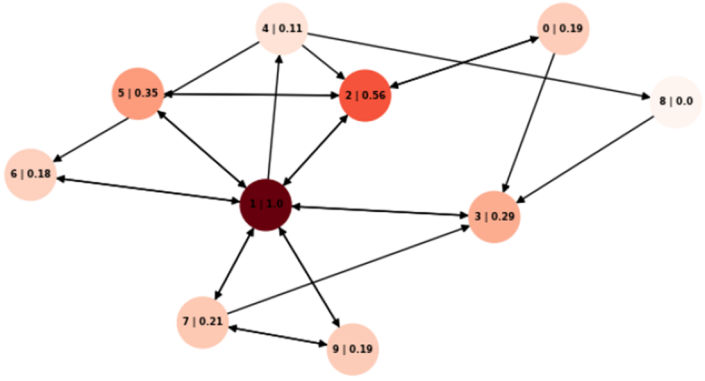
Several interesting Kernels can be used to prepare less common rankings, we have given some examples of such Kernels in the figures presented in Appendix A: Fig. 11 (cosine Kernel), Fig. 13 (monomial Kernel) and Fig. 12 (inverse Kernel). The cosine Kernel, $(cos(\lambda_i \pi/4)^t)_i$, with $t = 5$ in Fig. 11, provides a considerably different ranking where external nodes achieve significantly higher rankings, to the detriment of central nodes which seem to be penalized by this importance distribution. The inverse kernel behavior seems to be highly sensible to local graph structure: in Fig. 12, this phenomenon is highlighted by the fact that the right part of the importance distribution



(a) Generalized Importance Vector

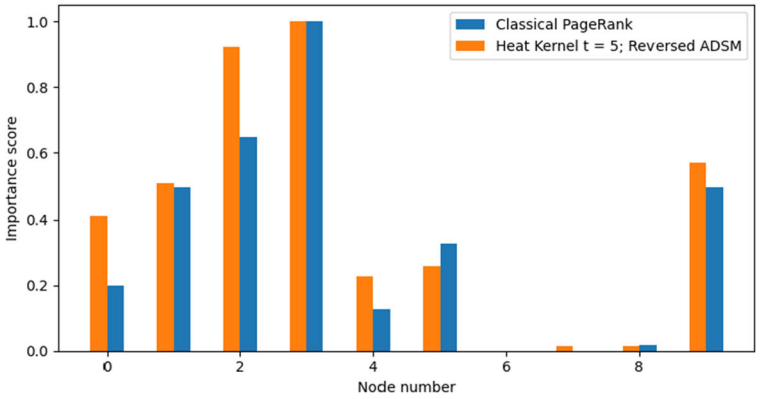


(b) Classical PageRank

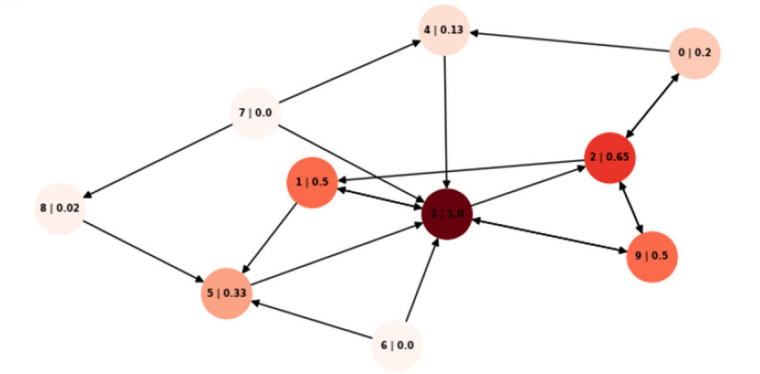


(c) Heat Kernel t=5, Reversed ADSM

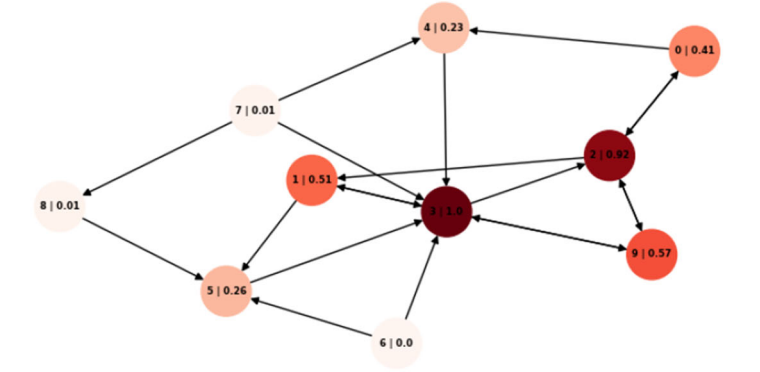
Fig. 4 Generalized PageRank Simulation Using a heat Kernel of parameter $t = 5$ for the Reversibilized version of the ADSM



(a) Generalized Importance Vector



(b) Classical PageRank



(c) Heat Kernel t=5, Reversed ADSM

Fig. 5 Generalized PageRank Simulation Using a heat Kernel of parameter $t = 5$ for the Reversibilized version of the ADSM of a simple graph

Table 1 Ranking comparison between the classical PageRank and the generalized PageRank using the Heat Kernel of parameter $t = 5$. Data from Fig. 4

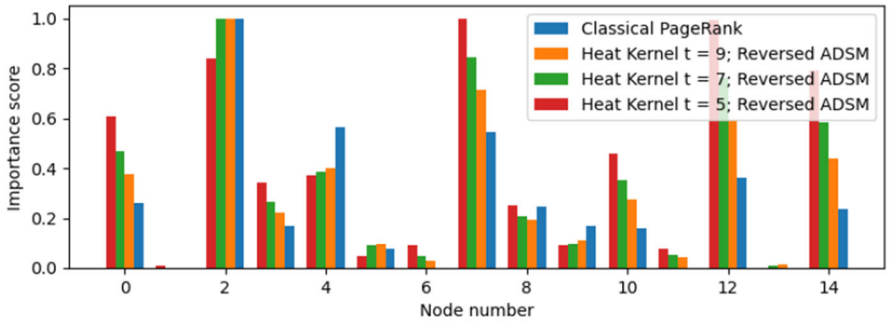
Ranking	Node Classical Kernel	Score Classical Kernel	Node Heat Kernel Time = 5	Score Heat Kernel Time = 5
1	1	1.000	1	1.000
2	2	0.348	2	0.560
3	3	0.338	5	0.347
4	5	0.210	3	0.294
5	7	0.198	7	0.206
6	9	0.168	9	0.193
7	6	0.135	0	0.191
8	4	0.081	6	0.178
9	0	0.076	4	0.111
10	8	0.000	8	0.000

Table 2 Ranking comparison between the classical PageRank and the generalized PageRank using the Heat Kernel of Parameter $t = 5$. Data from Fig. 5

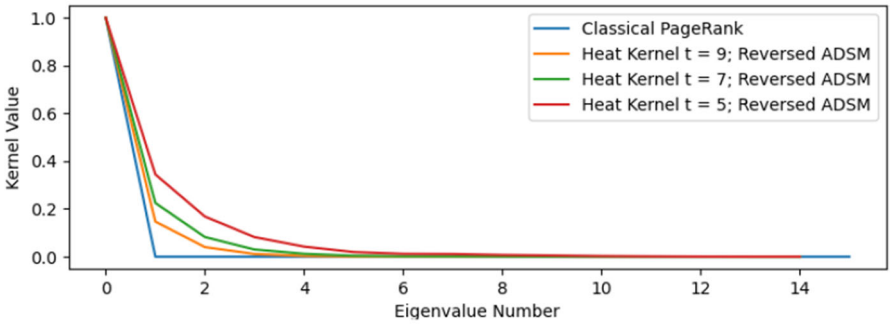
Ranking	Node Classical Kernel	Score Classical Kernel	Node Heat Kernel Time = 5	Score Heat Kernel Time = 5
1	3	1.000	3	1.000
2	2	0.647	2	0.923
3	9	0.500	9	0.571
4	1	0.500	1	0.510
5	5	0.326	0	0.411
6	0	0.200	5	0.259
7	4	0.126	4	0.227
8	8	0.017	8	0.014
9	6	0.000	7	0.014
10	7	0.000	6	0.000

graph absorbs most of the importance score for the inverse Heat Kernel, whereas this phenomenon does not appear in classical PageRank. Finally, the monomial Kernel is illustrated in Fig. 13, providing a more balanced ranking than the other kernels for the same parameter: the difference between the most important nodes and the secondary ones is less visible for this kind of importance distribution.

Table 3 provides a first benchmark for convergence analysis of ADSM using the power method. One notes that the number of iterations given a precision epsilon remains constant increasing the number of nodes of randomly generated Barabasi-Albert graphs. Adding this observation to the fact that ADSM are sparse matrices, computing the reversibilized version of an ADSM can be done in $\mathcal{O}(nnz(P))$ operations, which $\mathbf{P}$ the transition matrix of the input graph—which is at least as fast as the classical PageRank.



(a) Generalized Importance Vector



(b) Kernel distribution for generalized PageRank

Fig. 6 Generalized PageRank Simulation Using different heat Kernels for the Reversibilized version of the ADSM of the simple graph of Fig. 9

5.5 Comparison with Paparo's discrete PageRank

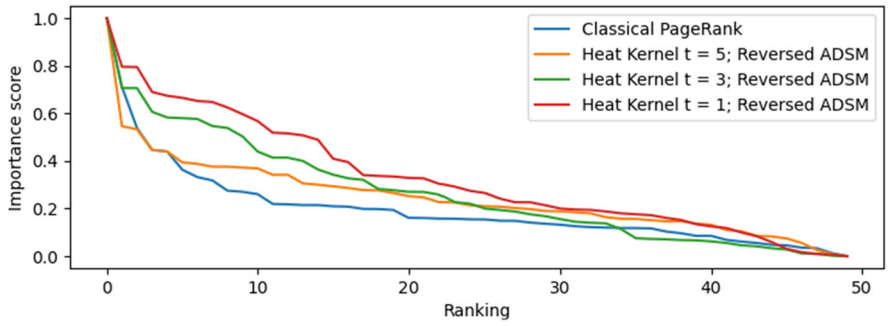
One can note several similarities between our Generalized Quantum PageRank and the one Paparo et al. introduced in 2012 in [11]. The authors derive the following expression for the instantaneous PageRank score:

$$I_q(P_i, m) = \| \sum_{\mu_{i,\pm}} \mu_{i,\pm}^{2m} \langle i|_2 |\mu_{i,\pm}\rangle \langle \mu_{i,\pm}|\psi(0)\rangle \|^2 \tag{27}$$

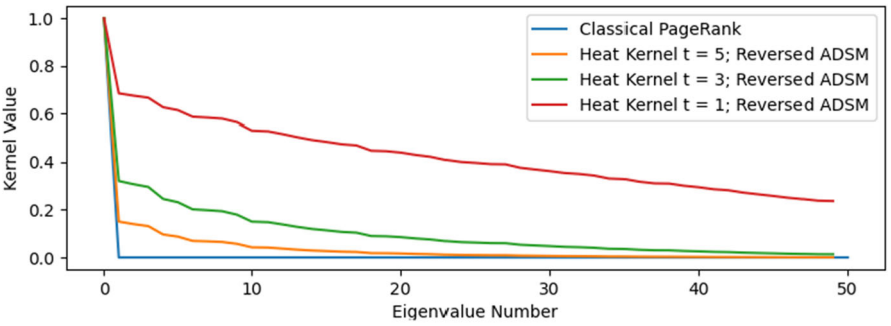
Where $|\psi(0)\rangle$ is an initial quantum state, $\mu_{i,\pm}$ are the eigenvalues of the Quantum Walk step operator that lie on the dynamical subspace (see Sect. 2) and $|\mu_{i,\pm}\rangle$ the associated eigenvector. Given the classical Google Matrix $\mathbf{G}$, if one notes $\mathbf{D}$ the discriminant matrix defined by $D_{ij} = \sqrt{G_{ij}G_{ji}}$, λ_i its eigenvalues and $|\lambda_i\rangle$ the associated eigenvectors, then Paparo et al. derive the following set of relations (valid on the dynamical subspace, where $\mathbf{U}^2$ the two-step walk operator acts non-trivially):

$$\mu_{j,\pm} = \exp(\pm i \, arcos(\lambda_j)) \tag{28}$$

$$|\mu_{i,\pm}\rangle = |\widetilde{\lambda_i}\rangle - \mu_{i,\pm} S|\widetilde{\lambda_i}\rangle \tag{29}$$



(a) Generalized Importance Vector



(b) Kernel distribution for generalized PageRank

Fig. 7 Generalized PageRank Simulation Using different heat Kernels for the Reversibilized version of the ADSM of the complex graph of Fig. 10

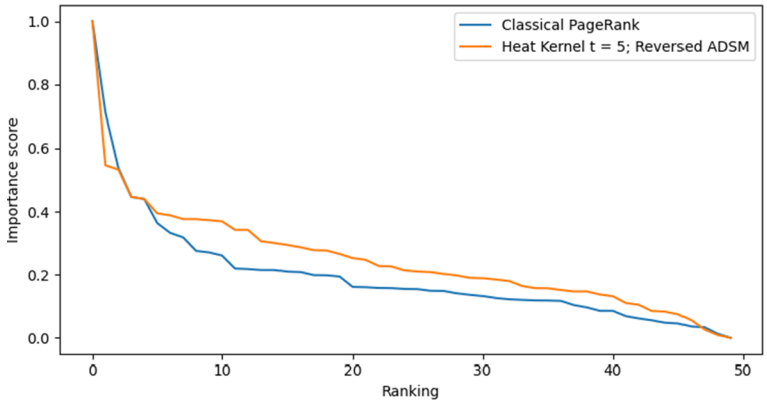
Where $|\widetilde{\lambda_i}\rangle = \mathbf{A}|\lambda_i\rangle$ where $\mathbf{A} = \sum_{j=1}^{N} |\psi_j\rangle\langle j|$ is an operator from the space of vertices $\mathbb{C}^N$, to the space of edges $\mathbb{C}^N \otimes \mathbb{C}^N$. The state $|\psi_j\rangle$ was defined in Sect. 2.

First note that one can apply directly our algorithm using a unitary encoding of $\mathbf{U}_{dyn}$ (the walk operator restricted to the dynamical subspace)—since $\mathbf{U}_{dyn}$ is unitary, its unitary encoding is itself. Then, using the singular value transformation to apply the polynomial $R_m = X^{2m}$ to the eigenspaces of $\mathbf{U}_{dyn}$, which allows one to get the final state:

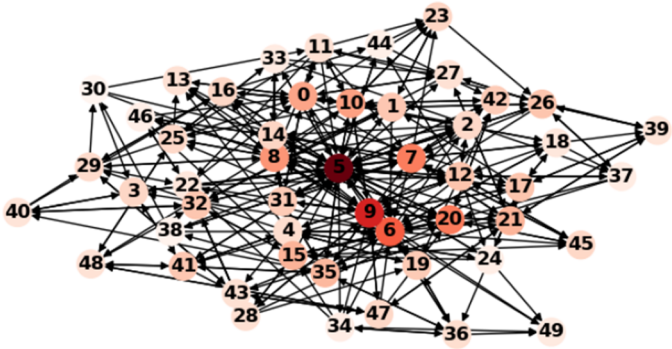
$$|\psi_f\rangle = \sum_{\mu_{i,\pm}} \mu_{i,\pm}^{2m} |\mu_{i,\pm}\rangle\langle\mu_{i,\pm}|\psi_0\rangle \tag{30}$$

Which exactly translates the expression of the instantaneous PageRank introduced above in 27. Indeed, let's remind that since $\mathbf{U}_{dyn}$ is unitary, $\mathbf{U}_{dyn}$ is normal and its left and right singular vectors coincide, which allows us to derive the above equation.

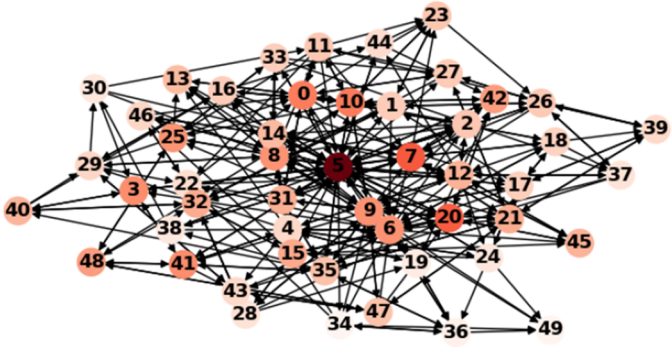
Note that, in Eq. 27, $\langle i|_2$ is applied only to the last n qubits of the space of the edges $\mathbb{C}^N \otimes \mathbb{C}^N$. Indeed, $\mathbf{U}_{dyn}$ is a $2N \times 2N$ matrix (see [11]). This feature is important in [11]: indeed, this phenomenon may explain some observed quantum effects on the final ranking—due to the non-trivial superposition of eigenstates represented by the sum of Eq. 27.



(a) Generalized Importance Vector



(b) Classical PageRank



(c) Heat Kernel t=5, Reversed ADSM

Fig. 8 Generalized PageRank Heat Kernel comparison with the Classical PageRank for a complex graph. Parameter $t = 5$

Table 3 Results of convergence benchmark for the ADSM. Precision 10^{-7}, graph type: Barabasi-Albert, parameter 5

Number of nodes	Number of iterations
100	21
215	22
464	22
1000	22
2154	21
4641	20
10000	20
21544	18
46415	22
100000	17

Computing the discrete quantum PageRank as Paparo et al. proposed in [11] becomes possible on a quantum computer providing that one can prepare the initial state $|\psi_0\rangle = \sum_i \mathbf{U}_{dyn}|i\rangle|j\rangle$ efficiently. One would have to repeat the measurements of each instantaneous PageRank instance by applying successively the singular value transformation with the polynomials $R_m = X^{2m}$. However, let's point out that the eigenvalues of the dynamical subspace are of the form $e^{i \arccos(\lambda_i)}$ which becomes $e^{2mi \arccos(\lambda_i)}$ when exponentiated using R_m.

Let's push the analogy with our method a bit further to try and understand the origin of quantum superposition that arises from the use of the walk operator $\mathbf{U}_{dyn}$. Indeed, one has:

$$|\psi_f\rangle = \sum_{i,\pm} \mu_{i,\pm}^{2m}(\mathbf{Id} - \mathbf{S}\mu_{i,\pm})|\widetilde{\lambda_i}\rangle\langle\mu_{i,\pm}|\psi(0)\rangle \tag{31}$$

$$= \sum_{i,\pm} \mu_{i,\pm}^{2m}|\widetilde{\lambda_i}\rangle\langle\widetilde{\lambda_i}|\psi(0)\rangle - \mathbf{S}\sum_{i,\pm} \mu_{i,\pm}^{2m+1}|\widetilde{\lambda_i}\rangle\langle\mu_{i,\pm}|\psi(0)\rangle \tag{32}$$

Let's choose the initial state $|\psi(0)\rangle = C\sum_i |\widetilde{\lambda_i}\rangle$, where C is a normalization constant we will drop to simplify the notations.

$$|\psi_f\rangle = \sum_{i,\pm}(\mu_{i,\pm}^{2m} - \lambda_i\mu_{i,\pm}^{2m+1})|\widetilde{\lambda_i}\rangle - \mathbf{S}\sum_{i,\pm}(\mu_{i,\pm}^{2m+1} - \lambda_i\mu_{i,\pm}^{2m+2}|\widetilde{\lambda_i}\rangle \tag{33}$$

$$= 2\sum_i (T_{2m}(\lambda_i) - \lambda_i T_{2m+1}(\lambda_i))|\widetilde{\lambda_i}\rangle$$

$$- 2\mathbf{S}\sum_i (T_{2m+1}(\lambda_i) - \lambda_i T_{2m+2}(\lambda_i))|\widetilde{\lambda_i}\rangle \tag{34}$$

$$\propto \sum_i (\lambda_i T_{2m+1}(\lambda_i) - T_{2m}(\lambda_i))|\widetilde{\lambda_i}\rangle - \mathbf{U}\sum_i (\lambda_i T_{2m+2}(\lambda_i)$$

$$- T_{2m+1}(\lambda_i))|\widetilde{\lambda_i}\rangle \tag{35}$$

In the last line, we have dropped the two normalization constants and used the fact that $\mathbf{U}|\widetilde{\lambda_i}\rangle = \mathbf{S}|\widetilde{\lambda_i}\rangle$ (see [11]).

T_k is the kth Chebyshev Polynomial of the first kind. We use the following relation to derive the second equality:

$$T_k(\lambda_i) = \frac{1}{2}\left(\left(\lambda_i + \sqrt{\lambda_i^2 - 1}\right)^k + \left(\lambda_i - \sqrt{\lambda_i^2 - 1}\right)^k\right) = \frac{1}{2}(\mu_{i,+} + \mu_{i,-}) \quad (36)$$

Equation 35 includes a linear combination of states that can be built using our Spectral Graph Theory quantum algorithm: $\sum_i(\lambda_i T_{2m+1}(\lambda_i) - T_{2m}(\lambda_i))|\widetilde{\lambda_i}\rangle$ and $\mathbf{U}\sum_i(\lambda_i T_{2m+2}(\lambda_i) - T_{2m+1}(\lambda_i))|\widetilde{\lambda_i}\rangle$.

Recently, the authors in [10], have provided an efficient method to emulate the polynomial T_{2m} in the singular value transformation, and the linear combination theorems of unitary block encoding. This method allows us to prepare efficiently the two former states in combination with our quantum algorithm. In this very special case, the amplitudes add up and create the quantum superposition effect exploited by Paparo et al. to compute the instantaneous PageRanks: one can see that the first member frequencies are affected by the higher frequencies of the second term. The second term is related to the first one as it is a rotation (by the unitary $\mathbf{U}$) on a higher degree filter of the eigenspaces of the discriminant matrix—in other words, using the Spectral Graph Theory framework, the instantaneous PageRank is derived from the interference of two Kernel generated graph signal states; one of which being rotated by the unitary $\mathbf{U}$.

This discussion paves the way to a generalized quantum based PageRank model, which adapts the Fourier Transform based generalized PageRank approach to the quantum domain.

Definition 5.4 Quantum Fourier Transform based PageRank Let $G = (V, E)$ a directed graph, $(\lambda_1, \ldots, \lambda_n)$ be a Fourier Basis associated with G, $(\mathbb{H}_k)_{k\in\mathbb{N}} = (h_{1,k}, \ldots, h_{n,k})$ a sequence of graph kernels with the associated coefficients $(\widehat{h}_{1,k}, \ldots, \widehat{h}_{n,k})$ of the inverse graph Fourier Transform. Let $|\widetilde{\lambda_i}\rangle$ be the vector state defined as above.

The (Instantaneous) Quantum Fourier Transform PageRank may be defined, like in [11], by:

$$\mathcal{I}(\mathbb{H}, k) = \||\psi_f\rangle\| = \|\sum_i \widehat{h}_{i,2k}|\widetilde{\lambda_i}\rangle - \mathbf{U}\sum_i \widehat{h}_{i,2k+1}|\widetilde{\lambda_i}\rangle\| \quad (37)$$

Note that if one chooses a sequence of Kernels $\mathbb{H}_k$, one can define a time averaged PageRank by:

$$\forall T \in \mathbb{N}, \mathcal{A}(\mathbb{H}, T) = \frac{1}{T}\sum_{i=1}^{T} \mathcal{I}_{\mathbb{H},i} \quad (38)$$

The study of this generalized quantum PageRank formulation is left as an open research topic.

6 Conclusion

Our paper presents three contributions to the field of quantum information processing with applications in web search engine algorithms:

- First, we introduce a Quantum Circuit-based algorithm, dealing with a class of problems similar to the one of Adiabatic Quantum algorithm introduced by [9]. Our circuit prepares the state $|\pi\rangle$, π being the classical Google Importance vector, simulating an Hamiltonian for time $\mathcal{O}(\frac{\log(N)}{\epsilon})$. We have then detailed several use cases for this algorithm.
- The main contribution of the paper consists in the introduction of one of the first Quantum Based Algorithms performing a wide range of graph signal filters and Kernel-based signal production systems with a low gate complexity. This algorithm relies heavily on the Quantum Singular Value Transformation, a recent field of Quantum Signal Processing.
- Our last contribution consists in applying the Spectral Graph Theory Framework to the PageRank algorithm theory in order to generalize the scope of the original algorithm. To this effect, we introduce a generalized PageRank ranking method based on the Directed Graph Fourier Transform which allows to produce more pertinent rankings by filtering the spectrum of the Google Graph. We then introduce an extension of the Google Matrix that can be used conjointly with the generalized PageRank algorithm to fully exploit its versatility. Finally, we provide an application example of our Spectral Graph Quantum Algorithm to the generalized PageRank model.

The work presented here, describing the links between Spectral Graph Theory, PageRank, and Quantum Information Processing, paves the way to a new, yet exciting extension of Quantum Signal Processing—the use of Quantum Algorithms in application to Spectral Graph Theory. To this extent, one of the most interesting questions raised here may be to research more in depth for the generalization of Quantum PageRank algorithms using our generalized PageRank approach.

Author Contributions T. Chapuis-Chkaiban, as the main author, produced most of the results presented in this paper and wrote the first version of the manuscript. Z.Toffano and B.Valiron, as co-authors, have made several research suggestions—both of them contributed equally to the scientific content. Besides, Z.Toffano and B.Valiron corrected and contributed to the writing and clarification of the manuscript and made several structural improvements for the final submitted version of this paper.

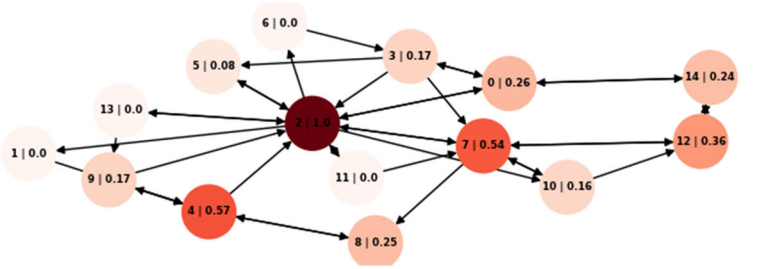
Data availability The datasets generated during and/or analyzed during the current study are available in the repository *Generalized PageRank* on Gitlab: https://gitlab-research.centralesupelec.fr/theodore.chapuis-chkaiban/generalised_pagerank

Declarations

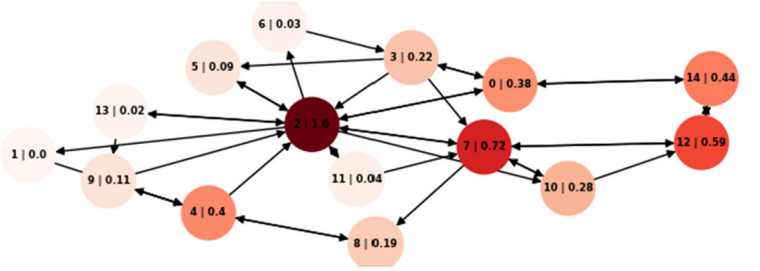
Conflict of interest The authors have no competing interests to declare that are relevant to the content of this article.

Appendix A Generalized PageRank using various Kernels

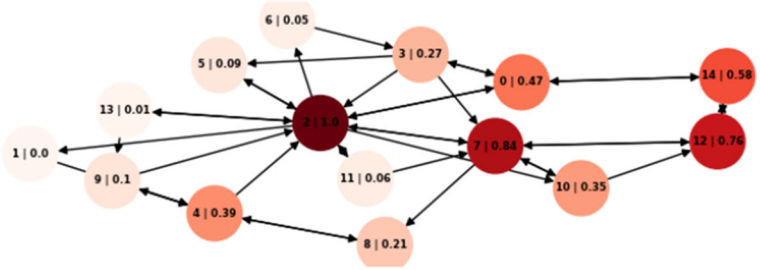
See Figs. 9, 10, 11, 12, 13 and Tables 4, 5.



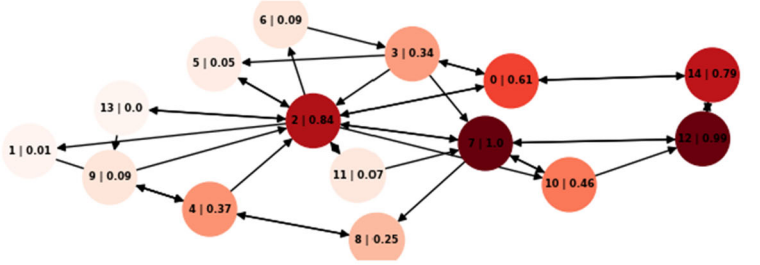
(a) Classical PageRank



(b) Heat Kernel t=9, Reversed ADSM



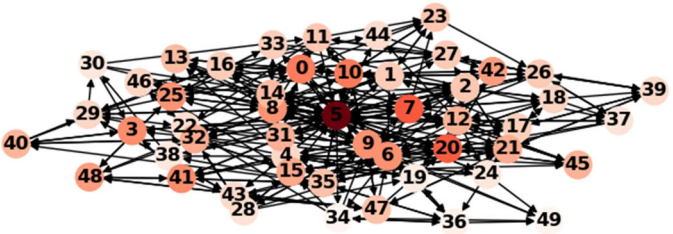
(c) Heat Kernel t=7, Reversed ADSM



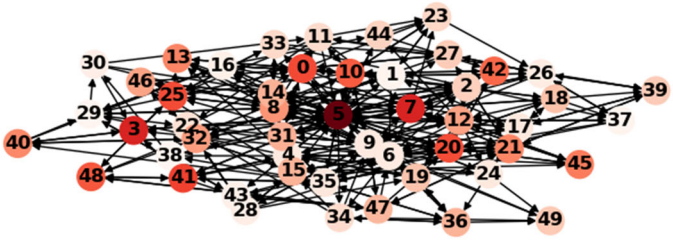
(d) Heat Kernel t=5, Reversed ADSM

Fig. 9 Generalized PageRank Simulation Using different heat Kernels for the Reversibilized version of the ADSM of a simple graph

(a) Classical PageRank



(b) Heat Kernel t=5, Reversed ADSM



(c) Heat Kernel t=3, Reversed ADSM

(d) Heat Kernel t=1, Reversed ADSM

Fig. 10 Generalized PageRank Simulation Using different heat Kernels for the reversibilized version of the ADSM of a complex graph

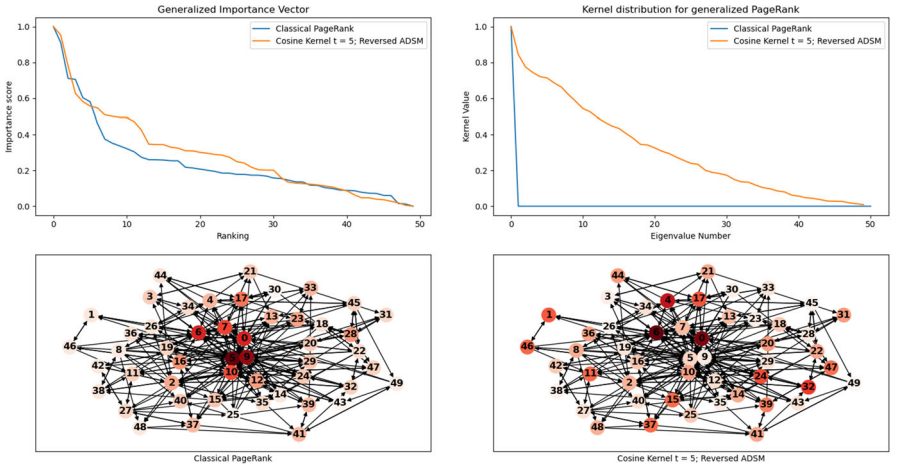


Fig. 11 Generalized PageRank Cosine Kernel comparison with the Classical PageRank. Parameter: exponent = 5

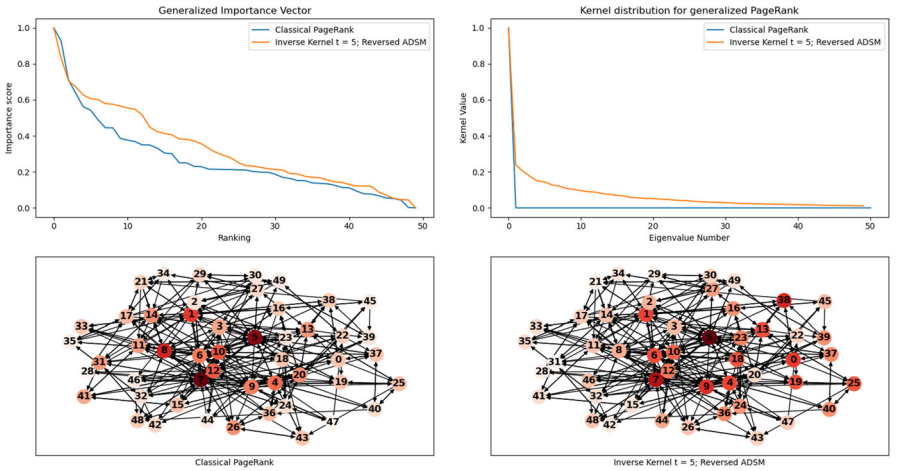


Fig. 12 Generalized PageRank Inverse Kernel comparison with the Classical PageRank. Parameter: exponent = 5

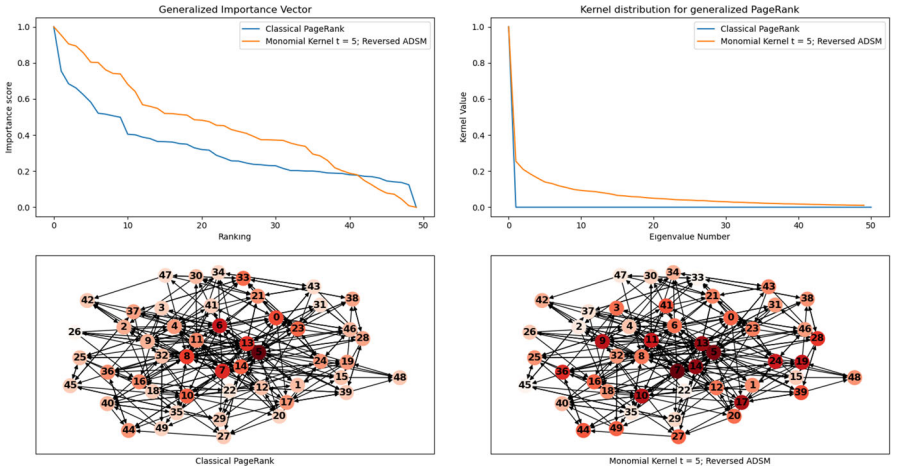


Fig. 13 Generalized PageRank monomial Kernel comparison with the Classical PageRank. Parameter: exponent = 5

Table 4 Ranking comparison between the classical Pagerank and the generalized pagerank using the Heat Kernels of parameters 5, 7 and 9

Nodes Ranking	Node Classical	Kernel Score Classical Kernel	Node Heat Kernel Time = 5	Score Heat Kernel Time = 5	Node Heat Kernel Time = 7	Score Heat Kernel Time = 7	Node Heat Kernel Time = 9	Score Heat Kernel Time = 9
1	2	1.000	7	1.000	2	1.000	2	1.000
2	4	0.566	12	0.993	7	0.843	7	0.715
3	7	0.544	2	0.839	12	0.760	12	0.591
4	12	0.363	14	0.793	14	0.582	14	0.438
5	0	0.260	0	0.607	0	0.469	4	0.400
6	8	0.245	10	0.457	4	0.387	0	0.377
7	14	0.238	4	0.371	10	0.350	10	0.276
8	9	0.171	3	0.343	3	0.266	3	0.233
9	3	0.167	8	0.253	8	0.209	8	0.193
10	10	0.157	9	0.093	9	0.099	9	0.109
11	5	0.077	6	0.092	5	0.090	5	0.094
12	1	0.000	11	0.075	11	0.055	11	0.043
13	6	0.000	5	0.048	6	0.046	6	0.028
14	11	0.000	1	0.007	13	0.011	13	0.017
15	13	0.000	13	0.000	1	0.000	1	0.000

Table 5 Ranking comparison between the classical Pagerank and the generalized pagerank using the Heat Kernel of parameter $t = 5$

Ranking	Node Classical Kernel	Score Classical Kernel	Node Heat Kernel Time = 5	Score Heat Kernel Time = 5
1	5	1.000	5	1.000
2	9	0.712	7	0.547
3	6	0.538	20	0.533
4	20	0.447	10	0.447
5	7	0.440	0	0.440
6	8	0.364	41	0.394
7	10	0.333	3	0.388
8	0	0.318	6	0.376
9	15	0.276	9	0.376
10	21	0.271	8	0.373
11	35	0.231	25	0.369
12	12	0.220	48	0.342
13	17	0.218	42	0.342
14	26	0.215	15	0.306
15	32	0.215	32	0.301
16	41	0.210	21	0.294
17	1	0.208	31	0.287
19	19	0.198	45	0.277
20	42	0.194	40	0.266
21	23	0.161	14	0.253
22	14	0.160	13	0.247
23	3	0.158	35	0.228
24	16	0.157	47	0.226
25	47	0.155	23	0.214
26	45	0.154	2	0.210
27	2	0.149	11	0.208
28	25	0.149	26	0.203
29	29	0.141	27	0.198
30	4	0.137	1	0.190
31	36	0.132	16	0.189
32	28	0.126	33	0.185
33	43	0.122	17	0.180
34	13	0.120	18	0.164
35	27	0.119	29	0.158
36	11	0.118	46	0.157
37	48	0.117	4	0.151
38	40	0.104	43	0.147
39	39	0.096	39	0.147

References

1. Brin, S., Page, L.: The anatomy of a large-scale hypertextual web search engine. Comput. Netw. ISDN Syst. **30**(1–7), 107–117 (1998). https://doi.org/10.1016/s0169-7552(98)00110-x
2. University, C.: Networks. https://blogs.cornell.edu/info2040/2014/11/03/more-than-just-a-web-search-algorithm-googles-pagerank-in-non-internet-contexts/
3. Gleich, D.F.: Pagerank beyond the web. SIAM Rev. **57**(3), 321–363 (2015). https://doi.org/10.1137/140976649
4. Shuman, D.I., Narang, S.K., Frossard, P., Ortega, A., Vandergheynst, P.: The emerging field of signal processing on graphs: Extending high-dimensional data analysis to networks and other irregular domains. IEEE Signal Process. Mag. **30**(3), 83–98 (2013). https://doi.org/10.1109/msp.2012.2235192
5. Chung, F.R.K., Graham, F.C., on Recent Advances in Spectral Graph Theory, C.C., (U.S.), N.S.F., Society, A.M., of the Mathematical Sciences, C.B.: Spectral graph theory (1997)
6. Ricaud, B., Borgnat, P., Tremblay, N., Gonçalves, P., Vandergheynst, P.: Fourier could be a data scientist: From graph Fourier transform to signal processing on graphs. C. R. Phys. **20**(5), 474–488 (2019). https://doi.org/10.1016/j.crhy.2019.08.003
7. Sevi, H., Rilling, G., Borgnat, P.: Harmonic analysis on directed graphs and applications: from Fourier analysis to wavelets (2019)
8. Harrow, A.W., Hassidim, A., Lloyd, S.: Quantum algorithm for linear systems of equations. Phys. Rev. Lett. **103**, 15 (2009). https://doi.org/10.1103/physrevlett.103.150502
9. Garnerone, S., Zanardi, P., Lidar, D.A.: Adiabatic quantum algorithm for search engine ranking. Phys. Rev. Lett. **108**, 23 (2012). https://doi.org/10.1103/physrevlett.108.230506
10. Gilyén, A., Su, Y., Low, G.H., Wiebe, N.: Quantum singular value transformation and beyond: exponential improvements for quantum matrix arithmetics. In: Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing (2019). https://doi.org/10.1145/3313276.3316366
11. Paparo, G.D., Martin-Delgado, M.A.: Google in a quantum network. Sci. Rep. **2**, 1 (2012). https://doi.org/10.1038/srep00444
12. Choromański, K., Matuszak, M., Jacek, M.: Scale-free graph with preferential attachment and evolving internal vertex structure. J. Stat. Phys. **151**(6), 1175–1183 (2013). https://doi.org/10.1007/s10955-013-0749-1
13. Barabási, A.-L., Albert, R.: Emergence of scaling in random networks. Science **286**(5439), 509–512 (1999). https://doi.org/10.1126/science.286.5439.509
14. Albert, R., Barabási, A.-L.: Statistical mechanics of complex networks. Rev. Mod. Phys. **74**(1), 47–97 (2002). https://doi.org/10.1103/revmodphys.74.47
15. Meyer, C.: Matrix analysis and applied linear algebra (2000). https://doi.org/10.1137/1.9780898719512
16. Boldi, P., Posenato, R., Santini, M., Vigna, S.: Traps and pitfalls of topic-biased pagerank. Algorithms and Models for the Web-Graph Lecture Notes in Computer Science, pp. 107–116. https://doi.org/10.1007/978-3-540-78808-9_10
17. Langville, A., Meyer, C.: Deeper inside pagerank. Internet Math. **1**(3), 335–380 (2004). https://doi.org/10.1080/15427951.2004.10129091
18. Paparo, G.D., Müller, M., Comellas, F., Martin-Delgado, M.A.: Quantum google algorithm. Eur. Phys. J. Plus **129**, 7 (2014). https://doi.org/10.1140/epjp/i2014-14150-y
19. Paparo, G.D., Müller, M., Comellas, F., Martin-Delgado, M.A.: Quantum google in a complex network. Sci. Rep. **3**, 1 (2013). https://doi.org/10.1038/srep02773
20. Sánchez-Burillo, E., Duch, J., Gómez-Gardeñes, J., Zueco, D.: Quantum navigation and ranking in complex networks. Sci. Rep. **2**, 1 (2012). https://doi.org/10.1038/srep00605
21. Loke, T., Tang, J.W., Rodriguez, J., Small, M., Wang, J.B.: Comparing classical and quantum pageranks. Quantum Inf. Process. **16**, 1 (2016). https://doi.org/10.1007/s11128-016-1456-z
22. Kempe, J.: Quantum random walks: an introductory overview. Contemp. Phys. **50**(1), 339–359 (2009). https://doi.org/10.1080/00107510902734722
23. Clader, B.D., Jacobs, B.C., Sprouse, C.R.: Preconditioned quantum linear system algorithm. Phys. Rev. Lett. **110**, 25 (2013). https://doi.org/10.1103/physrevlett.110.250504
24. Zhao, Z., Pozas-Kerstjens, A., Rebentrost, P., Wittek, P.: Bayesian deep learning on a quantum computer. Quantum Mach. Intell. **1**(1–2), 41–51 (2019). https://doi.org/10.1007/s42484-019-00004-7

25. Pan, J., Cao, Y., Yao, X., Li, Z., Ju, C., Chen, H., Peng, X., Kais, S., Du, J.: Experimental realization of quantum algorithm for solving linear systems of equations. Phys. Rev. A **89**, 2 (2014). https://doi.org/10.1103/physreva.89.022313
26. Team, T.Q.: Solving Linear Systems of Equations using HHL. Data 100 at UC Berkeley (2021). https://qiskit.org/textbook/ch-applications/hhl_tutorial.html
27. Grinko, D., Gacon, J., Zoufal, C., Woerner, S.: Iterative quantum amplitude estimation. NPJ Quantum Inf. **7**, 1 (2021). https://doi.org/10.1038/s41534-021-00379-1
28. Brassard, G., Høyer, P., Mosca, M., Tapp, A.: Quantum amplitude amplification and estimation. Contemp. Math. Quantum Comput. Inf. **305**, 53–74 (2002). https://doi.org/10.1090/conm/305/05215
29. Sandryhaila, A., Moura, J.M.F.: Discrete signal processing on graphs: Frequency analysis. IEEE Trans. Signal Process. **62**(12), 3042–3054 (2014). https://doi.org/10.1109/tsp.2014.2321121
30. Sardellitti, S., Barbarossa, S., Lorenzo, P.D.: Graph fourier transform for directed graphs based on lovász extension of min-cut. In: 2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP) (2017). https://doi.org/10.1109/icassp.2017.7952886
31. Magniez, F., Nayak, A., Roland, J., Santha, M.: Search via quantum walk. SIAM J. Comput. **40**(1), 142–164 (2011). https://doi.org/10.1137/090745854
32. Childs, A.M., Jeffery, S., Kothari, R., Magniez, F.: A time-efficient quantum walk for 3-distinctness using nested updates (2013). https://doi.org/10.48550/ARXIV.1302.7316
33. Lin, J., Bao, W.-S., Zhang, S., Li, T., Wang, X.: An improved quantum principal component analysis algorithm based on the quantum singular threshold method. Phys. Lett. A **383**(24), 2862–2868 (2019). https://doi.org/10.1016/j.physleta.2019.06.026
34. Duan, B., Yuan, J., Liu, Y., Li, D.: Efficient quantum circuit for singular-value thresholding. Phys. Rev. A **98**, 1 (2018). https://doi.org/10.1103/physreva.98.012308
35. Giovannetti, V., Lloyd, S., Maccone, L.: Architectures for a quantum random access memory. Phys. Rev. A **78**, 5 (2008). https://doi.org/10.1103/physreva.78.052310
36. Arunachalam, S., Gheorghiu, V., Jochym-O'Connor, T., Mosca, M., Srinivasan, P.V.: On the robustness of bucket brigade quantum ram. New J. Phys. **17**(12), 123010 (2015). https://doi.org/10.1088/1367-2630/17/12/123010
37. Park, D.K., Petruccione, F., Rhee, J.-K.K.: Circuit-based quantum random access memory for classical data. Sci. Rep. **9**, 1 (2019). https://doi.org/10.1038/s41598-019-40439-3
38. Szegedy, M.: Quantum speed-up of markov chain based algorithms. In: 45th Annual IEEE Symposium on Foundations of Computer Science. https://doi.org/10.1109/focs.2004.53
39. Ortega, A., Frossard, P., Kovacevic, J., Moura, J.M.F., Vandergheynst, P.: Graph signal processing: overview, challenges, and applications. Proc. IEEE **106**(5), 808–828 (2018). https://doi.org/10.1109/jproc.2018.2820126
40. Yang, H., King, I., Lyu, M.R.: Diffusionrank. In: Proceedings of the 30th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval - SIGIR 07 (2007). https://doi.org/10.1145/1277741.1277815
41. Baeza-Yates, R., Boldi, P., Castillo, C.: Generalizing pagerank. Proceedings of the 29th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval - SIGIR 06 (2006). https://doi.org/10.1145/1148170.1148225